

Complete Course: Deep Learning

From Neural Networks to Transformers

AI and Deep Learning Mastery

Contents

1	Introduction to Deep Learning	2
1.1	What is Deep Learning?	2
1.2	Applications of Deep Learning	3
2	Neural Network Fundamentals	3
2.1	The Artificial Neuron	3
2.2	Activation Functions	4
2.2.1	Sigmoid	5
2.2.2	Hyperbolic Tangent (tanh)	5
2.2.3	Rectified Linear Unit (ReLU)	5
2.2.4	Leaky ReLU	5
2.2.5	Softmax	6
2.3	Neural Network Architecture	9
2.4	Loss Functions	9
2.4.1	Mean Squared Error (MSE)	9
2.4.2	Binary Cross-Entropy	9
2.4.3	Categorical Cross-Entropy	10
2.5	Backpropagation Algorithm	10
2.5.1	Chain Rule in Backpropagation	11
2.5.2	Computational Graphs	11
2.6	Optimization Algorithms	19
2.6.1	Stochastic Gradient Descent (SGD)	19
2.6.2	Mini-batch Gradient Descent	19
2.6.3	Momentum	20
2.6.4	RMSProp	20
2.6.5	Adam (Adaptive Moment Estimation)	20
2.7	Regularization Techniques	25
2.7.1	L1/L2 Regularization	25
2.7.2	Dropout	25
2.7.3	Batch Normalization	26
2.7.4	Early Stopping	26
2.8	Weight Initialization	34

3	Convolutional Neural Networks (CNNs)	40
3.1	Introduction to CNNs	40
3.2	CNN Components	41
3.2.1	Convolutional Layer	41
3.2.2	Pooling Layer	41
3.2.3	Fully Connected Layer	41
3.3	CNN Architectures	42
3.3.1	LeNet-5 (1998)	42
3.3.2	AlexNet (2012)	42
3.3.3	VGGNet (2014)	42
3.3.4	ResNet (2015)	42
3.4	Advanced CNN Architectures	55
3.4.1	Inception Networks	55
3.4.2	DenseNet	55
3.4.3	EfficientNet	55
4	Recurrent Neural Networks (RNNs)	64
4.1	Introduction to Sequential Data	64
4.2	Vanilla RNN	65
4.3	Long Short-Term Memory (LSTM)	65
4.4	Gated Recurrent Unit (GRU)	66
5	Attention Mechanisms and Transformers	78
5.1	The Attention Mechanism	78
5.2	Self-Attention	78
5.3	Multi-Head Attention	78
5.4	Transformer Architecture	79
5.4.1	Positional Encoding	79
5.5	BERT and GPT Architectures	91
5.5.1	BERT (Bidirectional Encoder Representations from Transformers) . .	91
5.5.2	GPT (Generative Pre-trained Transformer)	92
6	Advanced Topics in Deep Learning	99
6.1	Generative Models	99
6.1.1	Variational Autoencoders (VAEs)	99
6.1.2	Generative Adversarial Networks (GANs)	99
6.2	Self-Supervised Learning	99
6.3	Model Optimization and Deployment	99
6.3.1	Quantization	99
6.3.2	Pruning	100
6.3.3	Knowledge Distillation	100
6.4	Ethical Considerations	100

7	Conclusion	100
7.1	Key Takeaways	100
7.2	Current Trends	101
7.3	Learning Resources	101
7.4	Final Advice	102

1 Introduction to Deep Learning

1.1 What is Deep Learning?

Definition 1.1 (Deep Learning). *Deep Learning is a subset of machine learning that uses artificial neural networks with multiple layers (deep architectures) to learn hierarchical representations from data.*

The Core Insight: Deep learning mimics the human brain's neural networks to process information in layers, where each layer extracts increasingly abstract features from raw data.

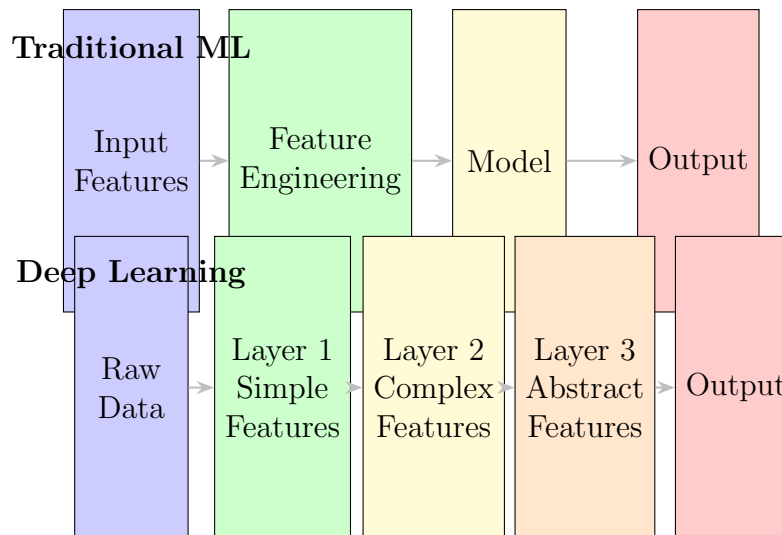


Figure 1: Comparison: Traditional ML requires manual feature engineering, while deep learning learns features automatically through layers

Why Deep Learning Now? Three key factors enabled the deep learning revolution:

1. **Big Data:** Availability of massive datasets (ImageNet: 14 million images)
2. **Computational Power:** GPUs and specialized hardware (TPUs)
3. **Algorithmic Advances:** New architectures and training techniques

1.2 Applications of Deep Learning

Domain		Applications	Key Architectures
Computer Vision	Vi-	Image classification, Object detection, Segmentation	CNNs, Vision Transformers
Natural Language Processing	Lan- guage Process-	Translation, Sentiment analysis, Chatbots	RNNs, Transformers, BERT
Speech Recognition	Recogni-	Voice assistants, Transcription	RNNs, CNNs, Transformers
Autonomous Vehicles	Ve-	Lane detection, Object tracking	CNNs, Reinforcement Learning
Healthcare		Disease diagnosis, Drug discovery	CNNs, GANs, Autoencoders

Table 1: Deep learning applications across domains

2 Neural Network Fundamentals

2.1 The Artificial Neuron

The Biological Inspiration: Neurons in the brain receive signals through dendrites, process them in the cell body, and send outputs through axons. Artificial neurons mimic this behavior mathematically.

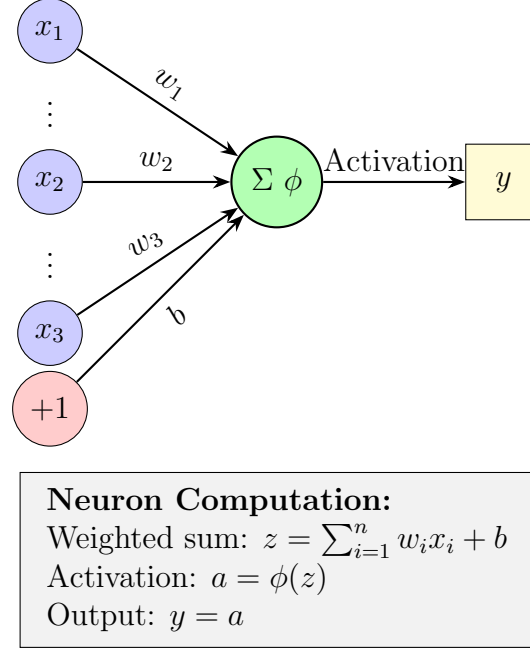


Figure 2: Artificial neuron: inputs are weighted, summed with bias, and passed through activation function

Definition 2.1 (Artificial Neuron). *An artificial neuron computes:*

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

where:

- x_i : Input features
- w_i : Weights (learnable parameters)
- b : Bias term (learnable parameter)
- ϕ : Activation function (non-linear transformation)

2.2 Activation Functions

Why Activation Functions? Without non-linear activation functions, neural networks would just be linear models regardless of depth.

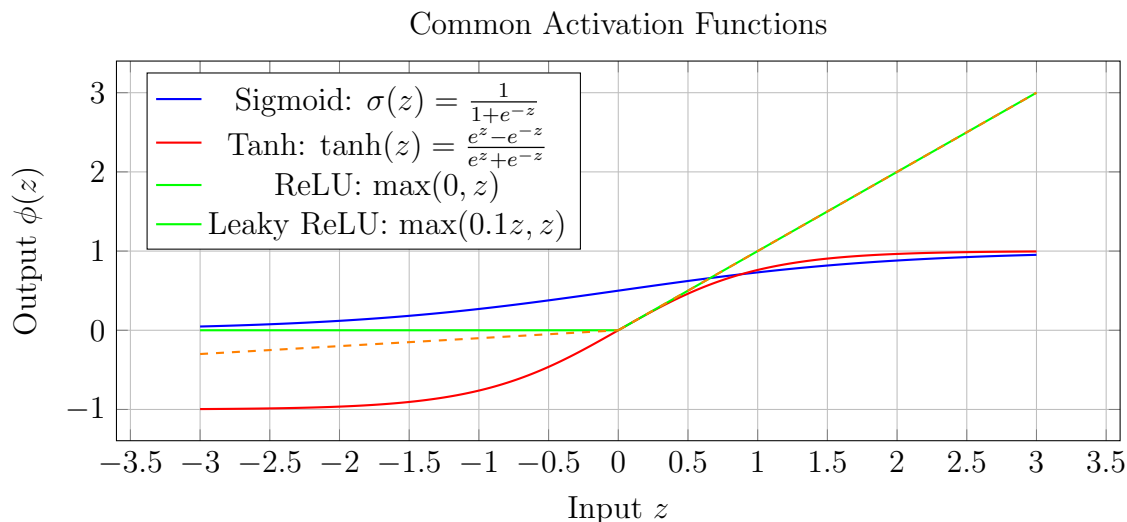


Figure 3: Non-linear activation functions enable neural networks to learn complex patterns

2.2.1 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{Output range: } (0, 1)$$

Usage: Output layer for binary classification (probability).

Problems: Vanishing gradients, not zero-centered, computationally expensive.

2.2.2 Hyperbolic Tangent (tanh)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \text{Output range: } (-1, 1)$$

Usage: Hidden layers, zero-centered helps optimization.

Problems: Still suffers from vanishing gradients.

2.2.3 Rectified Linear Unit (ReLU)

$$\text{ReLU}(z) = \max(0, z)$$

Usage: Default choice for hidden layers in most networks.

Advantages: Computationally efficient, reduces vanishing gradient.

Problems: "Dying ReLU" problem (neurons can get stuck at 0).

2.2.4 Leaky ReLU

$$\text{LeakyReLU}(z) = \max(\alpha z, z) \quad \text{where } \alpha \text{ is small (e.g., 0.01)}$$

Usage: Fixes dying ReLU problem, often better than ReLU.

2.2.5 Softmax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Usage: Output layer for multi-class classification (produces probability distribution).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class ActivationFunctions:
5     @staticmethod
6     def sigmoid(x):
7         """Sigmoid activation function."""
8         return 1 / (1 + np.exp(-x))
9
10    @staticmethod
11    def sigmoid_derivative(x):
12        """Derivative of sigmoid."""
13        s = ActivationFunctions.sigmoid(x)
14        return s * (1 - s)
15
16    @staticmethod
17    def relu(x):
18        """ReLU activation function."""
19        return np.maximum(0, x)
20
21    @staticmethod
22    def relu_derivative(x):
23        """Derivative of ReLU."""
24        return (x > 0).astype(float)
25
26    @staticmethod
27    def tanh(x):
28        """Hyperbolic tangent activation function."""
29        return np.tanh(x)
30
31    @staticmethod
32    def tanh_derivative(x):
33        """Derivative of tanh."""
34        return 1 - np.tanh(x)**2
35
36    @staticmethod
37    def leaky_relu(x, alpha=0.01):
38        """Leaky ReLU activation function."""
39        return np.where(x > 0, x, alpha * x)
40
41    @staticmethod
42    def leaky_relu_derivative(x, alpha=0.01):
```

```

43     """Derivative of Leaky ReLU."""
44     dx = np.ones_like(x)
45     dx[x < 0] = alpha
46     return dx
47
48 @staticmethod
49 def softmax(x):
50     """Softmax activation function."""
51     # Numerical stability: subtract max
52     exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
53     return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
54
55 @staticmethod
56 def plot_activations():
57     """Plot all activation functions."""
58     x = np.linspace(-3, 3, 1000)
59
60     fig, axes = plt.subplots(2, 3, figsize=(15, 8))
61
62     # Sigmoid
63     axes[0, 0].plot(x, ActivationFunctions.sigmoid(x), 'b-',
64                     linewidth=2)
65     axes[0, 0].plot(x, ActivationFunctions.sigmoid_derivative(x),
66                     'r--', linewidth=2)
67     axes[0, 0].set_title('Sigmoid (blue) and its Derivative (red)')
68     axes[0, 0].grid(True, alpha=0.3)
69     axes[0, 0].set_xlabel('Input')
70     axes[0, 0].set_ylabel('Output')
71
72     # ReLU
73     axes[0, 1].plot(x, ActivationFunctions.relu(x), 'b-',
74                     linewidth=2)
75     axes[0, 1].plot(x, ActivationFunctions.relu_derivative(x), 'r--',
76                     linewidth=2)
77     axes[0, 1].set_title('ReLU (blue) and its Derivative (red)')
78     axes[0, 1].grid(True, alpha=0.3)
79     axes[0, 1].set_xlabel('Input')
80
81     # Tanh
82     axes[0, 2].plot(x, ActivationFunctions.tanh(x), 'b-',
83                     linewidth=2)
84     axes[0, 2].plot(x, ActivationFunctions.tanh_derivative(x), 'r--',
85                     linewidth=2)
86     axes[0, 2].set_title('Tanh (blue) and its Derivative (red)')
87     axes[0, 2].grid(True, alpha=0.3)
88     axes[0, 2].set_xlabel('Input')

```

```

83
84 # Leaky ReLU
85 axes[1, 0].plot(x, ActivationFunctions.leaky_relu(x), 'b-',
86               linewidth=2)
87 axes[1, 0].plot(x, ActivationFunctions.leaky_relu_derivative
88               (x), 'r--', linewidth=2)
89 axes[1, 0].set_title('Leaky ReLU (blue) and its Derivative (
90               red)')
91 axes[1, 0].grid(True, alpha=0.3)
92 axes[1, 0].set_xlabel('Input')
93 axes[1, 0].set_ylabel('Output')
94
95 # Softmax (2D visualization)
96 x1 = np.linspace(-2, 2, 20)
97 x2 = np.linspace(-2, 2, 20)
98 X1, X2 = np.meshgrid(x1, x2)
99 X = np.stack([X1.flatten(), X2.flatten()], axis=1)
100
101 # Compute softmax outputs
102 softmax_output = ActivationFunctions.softmax(X)
103 prob_class1 = softmax_output[:, 0].reshape(X1.shape)
104
105 im = axes[1, 1].imshow(prob_class1, extent=[-2, 2, -2, 2],
106                       origin='lower', cmap='viridis')
107 axes[1, 1].set_title('Softmax: Probability of Class 1')
108 axes[1, 1].set_xlabel('Input 1')
109 axes[1, 1].set_ylabel('Input 2')
110 plt.colorbar(im, ax=axes[1, 1])
111
112 # Swish (modern activation)
113 axes[1, 2].plot(x, x * ActivationFunctions.sigmoid(x), 'b-',
114               linewidth=2)
115 axes[1, 2].set_title('Swish: x * sigmoid(x)')
116 axes[1, 2].grid(True, alpha=0.3)
117 axes[1, 2].set_xlabel('Input')
118 axes[1, 2].set_ylabel('Output')
119
120 plt.tight_layout()
121 plt.show()
122
123 # Plot activation functions
124 ActivationFunctions.plot_activations()

```

Listing 1: Activation Functions Implementation

2.3 Neural Network Architecture

From Single Neuron to Network: Connecting multiple neurons in layers creates a neural network that can learn complex functions.

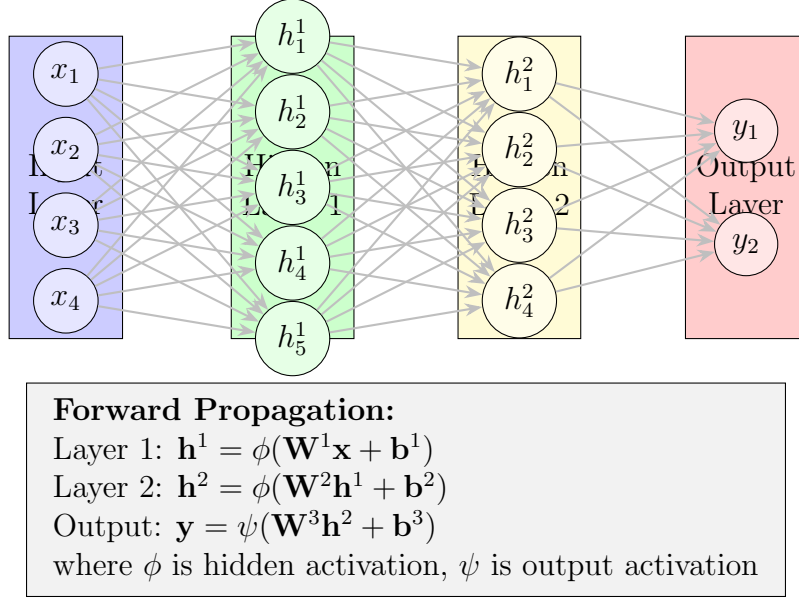


Figure 4: Multi-layer neural network with forward propagation equations

Definition 2.2 (Feedforward Neural Network). *A feedforward neural network with L layers computes:*

$$\mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}), \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

for $l = 1, \dots, L$, where $\mathbf{a}^{(0)} = \mathbf{x}$ is the input.

2.4 Loss Functions

Measuring Error: Loss functions quantify how wrong the network's predictions are compared to true values.

2.4.1 Mean Squared Error (MSE)

For regression problems:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2.4.2 Binary Cross-Entropy

For binary classification:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

2.4.3 Categorical Cross-Entropy

For multi-class classification:

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

2.5 Backpropagation Algorithm

The Learning Mechanism: Backpropagation efficiently computes gradients of the loss function with respect to all network parameters using the chain rule.

Algorithm 1 Backpropagation Algorithm

```
1: procedure BACKPROPAGATION( $X, y, \theta, \eta$ )
2:   # Forward pass
3:    $\mathbf{a}^{(0)} \leftarrow X$ 
4:   for  $l = 1$  to  $L$  do
5:      $\mathbf{z}^{(l)} \leftarrow \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ 
6:      $\mathbf{a}^{(l)} \leftarrow \phi^{(l)}(\mathbf{z}^{(l)})$ 
7:   end for
8:   Compute loss  $\mathcal{L}$ 
9:   # Backward pass
10:   $\delta^{(L)} \leftarrow \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \phi'(\mathbf{z}^{(L)})$ 
11:  for  $l = L - 1$  down to  $1$  do
12:     $\delta^{(l)} \leftarrow ((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}) \odot \phi'(\mathbf{z}^{(l)})$ 
13:  end for
14:  # Compute gradients
15:  for  $l = 1$  to  $L$  do
16:     $\nabla_{\mathbf{W}^{(l)}} \mathcal{L} \leftarrow \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$ 
17:     $\nabla_{\mathbf{b}^{(l)}} \mathcal{L} \leftarrow \delta^{(l)}$ 
18:  end for
19:  # Update parameters
20:  for  $l = 1$  to  $L$  do
21:     $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \nabla_{\mathbf{W}^{(l)}} \mathcal{L}$ 
22:     $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \nabla_{\mathbf{b}^{(l)}} \mathcal{L}$ 
23:  end for
24:  return updated  $\theta$ 
25: end procedure
```

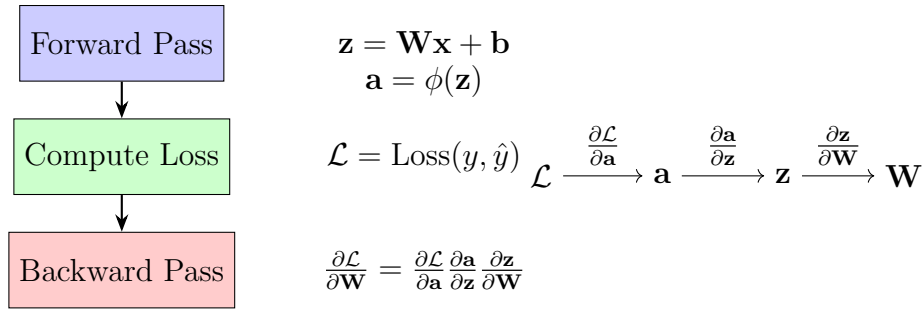


Figure 5: Backpropagation: Chain rule applied through computational graph

2.5.1 Chain Rule in Backpropagation

For a simple network $y = \phi(Wx + b)$:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial W}$$

where $z = Wx + b$.

2.5.2 Computational Graphs

Modern deep learning frameworks use computational graphs to automatically compute gradients via automatic differentiation.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class NeuralNetwork:
5     def __init__(self, layer_sizes, activations, learning_rate=0.01)
6         :
7         """
8         Initialize neural network.
9
10        Args:
11            layer_sizes: List of layer sizes [input_size,
12                        hidden1_size, ..., output_size]
13            activations: List of activation functions for each layer
14            learning_rate: Learning rate for gradient descent
15        """
16        self.layer_sizes = layer_sizes
17        self.activations = activations
18        self.learning_rate = learning_rate
19        self.L = len(layer_sizes) - 1 # Number of layers (excluding
20                                     input)
21
22        # Initialize parameters
23        self.weights = []

```

```

21     self.biases = []
22
23     # He initialization for ReLU, Xavier for sigmoid/tanh
24     for l in range(self.L):
25         fan_in = layer_sizes[l]
26         fan_out = layer_sizes[l+1]
27
28         if activations[l] == 'relu':
29             # He initialization
30             std = np.sqrt(2.0 / fan_in)
31         else:
32             # Xavier/Glorot initialization
33             std = np.sqrt(2.0 / (fan_in + fan_out))
34
35         W = np.random.randn(fan_out, fan_in) * std
36         b = np.zeros((fan_out, 1))
37
38         self.weights.append(W)
39         self.biases.append(b)
40
41     def forward(self, X):
42         """
43         Forward propagation through the network.
44
45         Args:
46             X: Input data of shape (input_size, batch_size)
47
48         Returns:
49             A: List of activations for each layer
50             Z: List of weighted sums for each layer
51         """
52         A = [X]    # Activations ( $a^0 = X$ )
53         Z = []     # Weighted sums
54
55         for l in range(self.L):
56             # Linear transformation
57             z = self.weights[l] @ A[-1] + self.biases[l]
58             Z.append(z)
59
60             # Activation function
61             if self.activations[l] == 'sigmoid':
62                 a = 1 / (1 + np.exp(-z))
63             elif self.activations[l] == 'relu':
64                 a = np.maximum(0, z)
65             elif self.activations[l] == 'tanh':
66                 a = np.tanh(z)
67             elif self.activations[l] == 'softmax':

```

```

68         # Stable softmax
69         exp_z = np.exp(z - np.max(z, axis=0, keepdims=True))
70         a = exp_z / np.sum(exp_z, axis=0, keepdims=True)
71     elif self.activations[1] == 'linear':
72         a = z
73     else:
74         raise ValueError(f"Unknown activation: {self.
75                             activations[1]}")
76
77     A.append(a)
78
79     return A, Z
80
81 def compute_loss(self, y_pred, y_true, loss_type='cross_entropy',
82 ):
83     """
84     Compute loss between predictions and true labels.
85
86     Args:
87         y_pred: Predictions
88         y_true: True labels
89         loss_type: Type of loss function
90
91     Returns:
92         Loss value
93     """
94     m = y_true.shape[1] # Batch size
95
96     if loss_type == 'cross_entropy':
97         # For classification (with softmax output)
98         # Clip predictions to avoid log(0)
99         y_pred = np.clip(y_pred, 1e-12, 1 - 1e-12)
100         loss = -np.sum(y_true * np.log(y_pred)) / m
101
102     elif loss_type == 'mse':
103         # For regression
104         loss = np.sum((y_pred - y_true)**2) / (2 * m)
105
106     elif loss_type == 'binary_cross_entropy':
107         # For binary classification
108         y_pred = np.clip(y_pred, 1e-12, 1 - 1e-12)
109         loss = -np.sum(y_true * np.log(y_pred) + (1 - y_true) *
110                         np.log(1 - y_pred)) / m
111
112     else:
113         raise ValueError(f"Unknown loss type: {loss_type}")

```

```

112     return loss
113
114 def backward(self, X, y, A, Z, loss_type='cross_entropy'):
115     """
116     Backward propagation to compute gradients.
117
118     Args:
119         X: Input data
120         y: True labels
121         A: List of activations from forward pass
122         Z: List of weighted sums from forward pass
123         loss_type: Type of loss function
124
125     Returns:
126         Gradients for weights and biases
127     """
128     m = X.shape[1] # Batch size
129     grads_W = []
130     grads_b = []
131
132     # Initialize gradients
133     dA = [None] * (self.L + 1) # Gradients for activations
134     dZ = [None] * self.L        # Gradients for weighted sums
135
136     # Output layer gradient
137     if loss_type == 'cross_entropy' and self.activations[-1] ==
138         'softmax':
139         # Special case: softmax + cross-entropy gradient
140         # simplifies
141         dA[-1] = A[-1] - y
142     else:
143         # General case: compute gradient through loss function
144         if loss_type == 'cross_entropy':
145             # dL/dy_pred = -y_true/y_pred
146             dL_dy_pred = -y / np.clip(A[-1], 1e-12, 1 - 1e-12)
147         elif loss_type == 'mse':
148             # dL/dy_pred = y_pred - y_true
149             dL_dy_pred = A[-1] - y
150         elif loss_type == 'binary_cross_entropy':
151             # dL/dy_pred = (y_pred - y_true) / (y_pred * (1 -
152             # y_pred))
153             dL_dy_pred = (A[-1] - y) / np.clip(A[-1] * (1 - A
154             [-1]), 1e-12, 1 - 1e-12)
155
156     # Activation derivative
157     if self.activations[-1] == 'sigmoid':
158         dA_dZ = A[-1] * (1 - A[-1])

```

```

155         elif self.activations[-1] == 'relu':
156             dA_dZ = (Z[-1] > 0).astype(float)
157         elif self.activations[-1] == 'tanh':
158             dA_dZ = 1 - A[-1]**2
159         elif self.activations[-1] == 'linear':
160             dA_dZ = 1
161         else:
162             raise ValueError(f"Unknown activation: {self.
163                               activations[-1]}")
164
165         dA[-1] = dL_dy_pred * dA_dZ
166
167     # Backpropagate through layers
168     for l in reversed(range(self.L)):
169         # Gradient for weighted sum
170         dZ[l] = dA[l+1]
171
172         # Gradients for parameters
173         grad_W = (dZ[l] @ A[l].T) / m
174         grad_b = np.sum(dZ[l], axis=1, keepdims=True) / m
175
176         grads_W.insert(0, grad_W)
177         grads_b.insert(0, grad_b)
178
179         # Gradient for previous layer's activation
180         if l > 0:
181             # Activation derivative for hidden layer
182             if self.activations[l-1] == 'sigmoid':
183                 dA_dZ_prev = A[l] * (1 - A[l])
184             elif self.activations[l-1] == 'relu':
185                 dA_dZ_prev = (Z[l-1] > 0).astype(float)
186             elif self.activations[l-1] == 'tanh':
187                 dA_dZ_prev = 1 - A[l]**2
188             elif self.activations[l-1] == 'linear':
189                 dA_dZ_prev = 1
190             else:
191                 raise ValueError(f"Unknown activation: {self.
192                                   activations[l-1]}")
193
194             # Backpropagate to previous layer
195             dA[l] = (self.weights[l].T @ dZ[l]) * dA_dZ_prev
196
197     return grads_W, grads_b
198
199 def update_parameters(self, grads_W, grads_b):
200     """
201     Update network parameters using gradients.

```

```

200
201     Args:
202         grads_W: Gradients for weights
203         grads_b: Gradients for biases
204     """
205     for l in range(self.L):
206         self.weights[l] -= self.learning_rate * grads_W[l]
207         self.biases[l] -= self.learning_rate * grads_b[l]
208
209     def train(self, X_train, y_train, X_val=None, y_val=None,
210               epochs=100, batch_size=32, verbose=True):
211         """
212         Train the neural network.
213
214         Args:
215             X_train: Training data
216             y_train: Training labels
217             X_val: Validation data (optional)
218             y_val: Validation labels (optional)
219             epochs: Number of training epochs
220             batch_size: Batch size for mini-batch gradient descent
221             verbose: Whether to print progress
222         """
223         n_samples = X_train.shape[1]
224         train_losses = []
225         val_losses = []
226
227         for epoch in range(epochs):
228             # Shuffle data
229             permutation = np.random.permutation(n_samples)
230             X_shuffled = X_train[:, permutation]
231             y_shuffled = y_train[:, permutation]
232
233             epoch_loss = 0
234
235             # Mini-batch training
236             for i in range(0, n_samples, batch_size):
237                 X_batch = X_shuffled[:, i:i+batch_size]
238                 y_batch = y_shuffled[:, i:i+batch_size]
239
240                 # Forward pass
241                 A, Z = self.forward(X_batch)
242
243                 # Compute loss
244                 batch_loss = self.compute_loss(A[-1], y_batch, '
                    cross_entropy')
245                 epoch_loss += batch_loss * X_batch.shape[1]

```

```

246         # Backward pass
247         grads_W, grads_b = self.backward(X_batch, y_batch, A
248             , Z, 'cross_entropy')
249
250         # Update parameters
251         self.update_parameters(grads_W, grads_b)
252
253         # Average loss for epoch
254         epoch_loss /= n_samples
255         train_losses.append(epoch_loss)
256
257         # Validation loss
258         if X_val is not None and y_val is not None:
259             A_val, _ = self.forward(X_val)
260             val_loss = self.compute_loss(A_val[-1], y_val, '
                cross_entropy')
261             val_losses.append(val_loss)
262
263         # Print progress
264         if verbose and (epoch + 1) % 10 == 0:
265             if X_val is not None:
266                 print(f"Epoch {epoch+1}/{epochs}, Train Loss: {
                    epoch_loss:.4f}, Val Loss: {val_loss:.4f}")
267             else:
268                 print(f"Epoch {epoch+1}/{epochs}, Train Loss: {
                    epoch_loss:.4f}")
269
270         return train_losses, val_losses
271
272     def predict(self, X):
273         """
274         Make predictions for input data.
275
276         Args:
277             X: Input data
278
279         Returns:
280             Predictions
281         """
282         A, _ = self.forward(X)
283         return A[-1]
284
285     def accuracy(self, X, y):
286         """
287         Compute accuracy for classification.
288

```

```

289     Args:
290         X: Input data
291         y: True labels (one-hot encoded)
292
293     Returns:
294         Accuracy
295     """
296     predictions = self.predict(X)
297     predicted_classes = np.argmax(predictions, axis=0)
298     true_classes = np.argmax(y, axis=0)
299     accuracy = np.mean(predicted_classes == true_classes)
300     return accuracy
301
302 # Example usage
303 if __name__ == "__main__":
304     # Generate synthetic dataset
305     np.random.seed(42)
306
307     # XOR problem (non-linearly separable)
308     X = np.array([[0, 0, 1, 1],
309                  [0, 1, 0, 1]])
310     y = np.array([[0, 1, 1, 0]]) # XOR output
311
312     # One-hot encode for classification
313     y_onehot = np.zeros((2, 4))
314     y_onehot[0, y[0] == 0] = 1
315     y_onehot[1, y[0] == 1] = 1
316
317     print("XOR Problem:")
318     print("X =", X.T)
319     print("y =", y.T)
320
321     # Create and train neural network
322     nn = NeuralNetwork(
323         layer_sizes=[2, 4, 2], # Input: 2, Hidden: 4, Output: 2
324         activations=['relu', 'softmax'], # Hidden: ReLU, Output:
325             Softmax
326         learning_rate=0.1
327     )
328
329     # Train
330     train_losses, _ = nn.train(
331         X_train=X,
332         y_train=y_onehot,
333         epochs=1000,
334         batch_size=4,
335         verbose=False

```

```

335     )
336
337     # Predict
338     predictions = nn.predict(X)
339     predicted_classes = np.argmax(predictions, axis=0)
340
341     print("\nPredictions:")
342     for i in range(X.shape[1]):
343         print(f"Input: {X[:, i]}, True: {y[0, i]}, Predicted: {
344             predicted_classes[i]}, "
345             f"Probabilities: {predictions[:, i].round(3)}")
346
347     print(f"\nAccuracy: {nn.accuracy(X, y_onehot):.2%}")
348
349     # Plot training loss
350     plt.figure(figsize=(10, 4))
351     plt.plot(train_losses)
352     plt.xlabel('Epoch')
353     plt.ylabel('Loss')
354     plt.title('Training Loss')
355     plt.grid(True, alpha=0.3)
356     plt.show()

```

Listing 2: Neural Network Implementation from Scratch

2.6 Optimization Algorithms

Beyond Vanilla Gradient Descent: Modern optimizers improve training speed and stability.

2.6.1 Stochastic Gradient Descent (SGD)

Update weights after each training example:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(x_i, y_i; \theta)$$

2.6.2 Mini-batch Gradient Descent

Compromise between SGD and batch GD:

$$\theta \leftarrow \theta - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \mathcal{L}(x_i, y_i; \theta)$$

where m is batch size (typically 32, 64, 128, 256).

2.6.3 Momentum

Accumulates gradient from previous steps to accelerate:

$$\begin{aligned}v_t &\leftarrow \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta) \\ \theta &\leftarrow \theta - v_t\end{aligned}$$

where γ is momentum coefficient (typically 0.9).

2.6.4 RMSProp

Adapts learning rate per parameter based on recent gradients:

$$\begin{aligned}s_t &\leftarrow \beta s_{t-1} + (1 - \beta)(\nabla_{\theta} \mathcal{L}(\theta))^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{s_t + \epsilon}} \nabla_{\theta} \mathcal{L}(\theta)\end{aligned}$$

2.6.5 Adam (Adaptive Moment Estimation)

Combines momentum and RMSProp:

$$\begin{aligned}m_t &\leftarrow \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta) && \text{(First moment)} \\ v_t &\leftarrow \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta))^2 && \text{(Second moment)} \\ \hat{m}_t &\leftarrow \frac{m_t}{1 - \beta_1^t} && \text{(Bias correction)} \\ \hat{v}_t &\leftarrow \frac{v_t}{1 - \beta_2^t} && \text{(Bias correction)} \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t\end{aligned}$$

Default: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

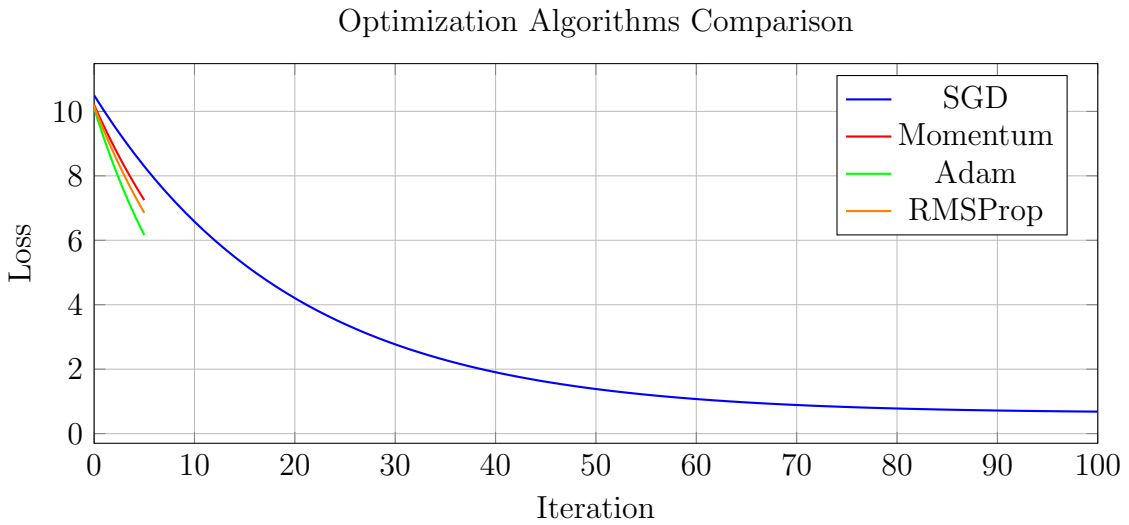


Figure 6: Optimization algorithms: Adam typically converges faster and more stably

```

1 import numpy as np
2
3 class Optimizer:
4     """Base optimizer class."""
5
6     def __init__(self, learning_rate=0.01):
7         self.learning_rate = learning_rate
8         self.t = 0 # Time step for Adam
9
10    def update(self, params, grads):
11        """Update parameters. To be implemented by subclasses."""
12        raise NotImplementedError
13
14
15    class SGD(Optimizer):
16        """Stochastic Gradient Descent."""
17
18        def __init__(self, learning_rate=0.01, momentum=0.0):
19            super().__init__(learning_rate)
20            self.momentum = momentum
21            self.velocity = None
22
23        def update(self, params, grads):
24            """Update parameters with SGD (optionally with momentum)."""
25            if self.velocity is None:
26                self.velocity = [np.zeros_like(p) for p in params]
27
28            updated_params = []
29            for i, (param, grad, vel) in enumerate(zip(params, grads,
30                self.velocity)):
31                # Update velocity
32                self.velocity[i] = self.momentum * vel - self.
33                    learning_rate * grad
34                # Update parameter
35                updated_params.append(param + self.velocity[i])
36
37            return updated_params
38
39    class Adam(Optimizer):
40        """Adam optimizer."""
41
42        def __init__(self, learning_rate=0.001, beta1=0.9, beta2=0.999,
43            epsilon=1e-8):
44            super().__init__(learning_rate)
45            self.beta1 = beta1
46            self.beta2 = beta2

```

```

45     self.epsilon = epsilon
46     self.m = None    # First moment
47     self.v = None    # Second moment
48     self.t = 0       # Time step
49
50     def update(self, params, grads):
51         """Update parameters with Adam."""
52         if self.m is None:
53             self.m = [np.zeros_like(p) for p in params]
54             self.v = [np.zeros_like(p) for p in params]
55
56         self.t += 1
57         updated_params = []
58
59         for i, (param, grad) in enumerate(zip(params, grads)):
60             # Update biased first moment estimate
61             self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) *
                grad
62
63             # Update biased second moment estimate
64             self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) *
                (grad ** 2)
65
66             # Compute bias-corrected first moment estimate
67             m_hat = self.m[i] / (1 - self.beta1 ** self.t)
68
69             # Compute bias-corrected second moment estimate
70             v_hat = self.v[i] / (1 - self.beta2 ** self.t)
71
72             # Update parameters
73             updated_param = param - self.learning_rate * m_hat / (np
                .sqrt(v_hat) + self.epsilon)
74             updated_params.append(updated_param)
75
76         return updated_params
77
78
79     class RMSprop(Optimizer):
80         """RMSprop optimizer."""
81
82         def __init__(self, learning_rate=0.001, rho=0.9, epsilon=1e-8):
83             super().__init__(learning_rate)
84             self.rho = rho
85             self.epsilon = epsilon
86             self.cache = None    # Running average of squared gradients
87
88         def update(self, params, grads):

```

```

89     """Update parameters with RMSprop."""
90     if self.cache is None:
91         self.cache = [np.zeros_like(p) for p in params]
92
93     updated_params = []
94
95     for i, (param, grad) in enumerate(zip(params, grads)):
96         # Update running average of squared gradients
97         self.cache[i] = self.rho * self.cache[i] + (1 - self.rho
98             ) * (grad ** 2)
99
100        # Update parameters
101        updated_param = param - self.learning_rate * grad / (np.
102            sqrt(self.cache[i]) + self.epsilon)
103        updated_params.append(updated_param)
104
105    return updated_params
106
107class Adagrad(Optimizer):
108    """Adagrad optimizer."""
109
110    def __init__(self, learning_rate=0.01, epsilon=1e-8):
111        super().__init__(learning_rate)
112        self.epsilon = epsilon
113        self.cache = None # Sum of squared gradients
114
115    def update(self, params, grads):
116        """Update parameters with Adagrad."""
117        if self.cache is None:
118            self.cache = [np.zeros_like(p) for p in params]
119
120        updated_params = []
121
122        for i, (param, grad) in enumerate(zip(params, grads)):
123            # Accumulate squared gradients
124            self.cache[i] += grad ** 2
125
126            # Update parameters
127            updated_param = param - self.learning_rate * grad / (np.
128                sqrt(self.cache[i]) + self.epsilon)
129            updated_params.append(updated_param)
130
131        return updated_params
132
133# Example: Comparing optimizers

```

```

133 def compare_optimizers():
134     """Compare different optimizers on a simple convex function."""
135
136     # Define a simple convex function:  $f(x) = x^2$ 
137     def f(x):
138         return x**2
139
140     def grad_f(x):
141         return 2*x
142
143     # Parameters for optimization
144     x0 = 10.0 # Starting point
145     epochs = 100
146
147     # Different optimizers
148     optimizers = {
149         'SGD': SGD(learning_rate=0.1),
150         'SGD with Momentum': SGD(learning_rate=0.1, momentum=0.9),
151         'Adam': Adam(learning_rate=0.1),
152         'RMSprop': RMSprop(learning_rate=0.1),
153         'Adagrad': Adagrad(learning_rate=0.5)
154     }
155
156     # Store results
157     results = {}
158
159     plt.figure(figsize=(12, 8))
160
161     for name, optimizer in optimizers.items():
162         x = x0
163         trajectory = [x]
164
165         for epoch in range(epochs):
166             # Compute gradient
167             grad = grad_f(x)
168
169             # Update parameter (wrap in lists for optimizer
170             # interface)
171             params = [np.array([x])]
172             grads = [np.array([grad])]
173
174             updated_params = optimizer.update(params, grads)
175             x = updated_params[0][0]
176             trajectory.append(x)
177
178             results[name] = trajectory

```

```

179     # Plot
180     plt.plot(trajecory, label=name, linewidth=2)
181     plt.scatter([len(trajecory)-1], [trajecory[-1]], s=50)
182
183     plt.axhline(y=0, color='black', linestyle='--', alpha=0.5, label
184               ='Optimum')
185     plt.xlabel('Iteration')
186     plt.ylabel('Parameter value (x)')
187     plt.title('Optimizer Comparison on f(x) = x^2$')
188     plt.legend()
189     plt.grid(True, alpha=0.3)
190     plt.show()
191
192     # Print final values
193     print("\nFinal parameter values after 100 iterations:")
194     for name, trajectory in results.items():
195         print(f"{name}: {trajectory[-1]:.6f}")
196
197     return results
198
199 # Run comparison
200 compare_optimizers()

```

Listing 3: Optimizer Implementation

2.7 Regularization Techniques

Preventing Overfitting: Deep neural networks have millions of parameters and can easily overfit to training data.

2.7.1 L1/L2 Regularization

Add penalty to loss function:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \sum_i |w_i| \quad (\text{L1})$$

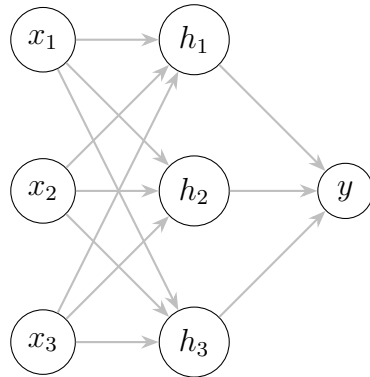
$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \sum_i w_i^2 \quad (\text{L2})$$

2.7.2 Dropout

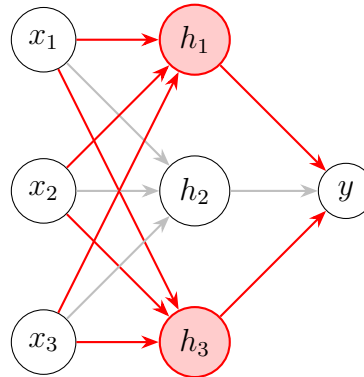
Randomly "drop" neurons during training with probability p :

- During training: Each neuron retained with probability $1 - p$
- During testing: Use all neurons, scale weights by $1 - p$

Original Network



With Dropout (p=0.5)

**Dropout Mathematics:**

Training: $h_i^{\text{dropout}} = r_i * h_i$ where $r_i \sim \text{Bernoulli}(1 - p)$

Testing: $h_i^{\text{test}} = (1 - p) * h_i$

Effect: Forces network to learn robust features that work with any subset

Figure 7: Dropout: Randomly deactivate neurons during training to prevent co-adaptation

2.7.3 Batch Normalization

Normalize activations of each layer:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (\text{Normalize})$$

$$y = \gamma \hat{x} + \beta \quad (\text{Scale and shift})$$

where γ, β are learnable parameters.

2.7.4 Early Stopping

Stop training when validation loss stops improving.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class Regularization:
5     """Implementation of various regularization techniques."""
6
7     @staticmethod
8     def l2_regularization(weights, lambda_reg):
9         """
10         Compute L2 regularization penalty and gradient.
11
12         Args:

```

```

13         weights: List of weight matrices
14         lambda_reg: Regularization strength
15
16     Returns:
17         penalty: L2 penalty value
18         gradients: Gradients for regularization term
19     """
20     penalty = 0
21     reg_gradients = []
22
23     for W in weights:
24         penalty += 0.5 * lambda_reg * np.sum(W ** 2)
25         reg_gradients.append(lambda_reg * W)
26
27     return penalty, reg_gradients
28
29 @staticmethod
30 def l1_regularization(weights, lambda_reg):
31     """
32     Compute L1 regularization penalty and gradient.
33
34     Args:
35         weights: List of weight matrices
36         lambda_reg: Regularization strength
37
38     Returns:
39         penalty: L1 penalty value
40         gradients: Gradients for regularization term
41     """
42     penalty = 0
43     reg_gradients = []
44
45     for W in weights:
46         penalty += lambda_reg * np.sum(np.abs(W))
47         reg_gradients.append(lambda_reg * np.sign(W))
48
49     return penalty, reg_gradients
50
51 @staticmethod
52 def dropout_forward(X, dropout_rate, mode='train'):
53     """
54     Apply dropout during forward pass.
55
56     Args:
57         X: Input activations
58         dropout_rate: Probability of dropping a neuron
59         mode: 'train' or 'test'

```

```

60
61     Returns:
62         Output with dropout applied (and mask for backward pass)
63     """
64     if mode == 'train':
65         # Create dropout mask
66         keep_prob = 1 - dropout_rate
67         mask = (np.random.rand(*X.shape) < keep_prob) /
68             keep_prob
69         return X * mask, mask
70     else:
71         # During test time, scale by keep probability
72         return X * (1 - dropout_rate), None
73
74 @staticmethod
75 def dropout_backward(dA, mask, dropout_rate):
76     """
77     Apply dropout during backward pass.
78
79     Args:
80         dA: Gradient from next layer
81         mask: Dropout mask from forward pass
82         dropout_rate: Dropout rate used
83
84     Returns:
85         Gradient with dropout applied
86     """
87     if mask is not None:
88         return dA * mask
89     else:
90         return dA * (1 - dropout_rate)
91
92 @staticmethod
93 def batch_normalization_forward(X, gamma, beta, eps=1e-8, mode='
94     train',
95                                     running_mean=None, running_var=
96                                     None, momentum=0.9):
97     """
98     Batch normalization forward pass.
99
100     Args:
101         X: Input of shape (D, N) where D is dimension, N is
102             batch size
103         gamma: Scale parameter
104         beta: Shift parameter
105         eps: Small constant for numerical stability
106         mode: 'train' or 'test'

```

```

103         running_mean: Running mean for inference
104         running_var: Running variance for inference
105         momentum: Momentum for updating running statistics
106
107     Returns:
108         Normalized output, cache for backward pass
109     """
110     if mode == 'train':
111         # Compute batch statistics
112         mu = np.mean(X, axis=1, keepdims=True) # Mean over
113         batch
114         var = np.var(X, axis=1, keepdims=True) # Variance over
115         batch
116
117         # Normalize
118         X_hat = (X - mu) / np.sqrt(var + eps)
119
120         # Scale and shift
121         out = gamma * X_hat + beta
122
123         # Update running statistics for inference
124         if running_mean is not None:
125             running_mean = momentum * running_mean + (1 -
126                 momentum) * mu
127             running_var = momentum * running_var + (1 - momentum
128                 ) * var
129
130         # Cache for backward pass
131         cache = (X, X_hat, mu, var, gamma, eps)
132
133         return out, cache, running_mean, running_var
134
135     else: # mode == 'test'
136         # Use running statistics for inference
137         X_hat = (X - running_mean) / np.sqrt(running_var + eps)
138         out = gamma * X_hat + beta
139         return out, None, running_mean, running_var
140
141 @staticmethod
142 def batch_normalization_backward(dout, cache):
143     """
144     Batch normalization backward pass.
145
146     Args:
147         dout: Gradient from next layer
148         cache: Cache from forward pass

```

```

146     Returns:
147         Gradients for X, gamma, beta
148     """
149     X, X_hat, mu, var, gamma, eps = cache
150     N = X.shape[1]
151
152     # Gradients for scale and shift
153     dgamma = np.sum(dout * X_hat, axis=1, keepdims=True)
154     dbeta = np.sum(dout, axis=1, keepdims=True)
155
156     # Gradient for normalized input
157     dX_hat = dout * gamma
158
159     # Gradient for variance
160     dvar = np.sum(dX_hat * (X - mu) * -0.5 * (var + eps) **
161                   -1.5, axis=1, keepdims=True)
162
163     # Gradient for mean
164     dmu = np.sum(dX_hat * -1 / np.sqrt(var + eps), axis=1,
165                  keepdims=True) + \
166           dvar * np.sum(-2 * (X - mu), axis=1, keepdims=True) /
167           N
168
169     # Gradient for input
170     dX = dX_hat / np.sqrt(var + eps) + \
171          dvar * 2 * (X - mu) / N + \
172          dmu / N
173
174     return dX, dgamma, dbeta
175
176 @staticmethod
177 def early_stopping(train_losses, val_losses, patience=10):
178     """
179     Determine if training should stop early.
180
181     Args:
182         train_losses: List of training losses
183         val_losses: List of validation losses
184         patience: Number of epochs to wait for improvement
185
186     Returns:
187         True if should stop, False otherwise
188     """
189     if len(val_losses) < patience + 1:
190         return False
191
192     # Check if validation loss hasn't improved for 'patience'

```

```

190         epochs
191         best_val_loss = min(val_losses)
192         recent_val_losses = val_losses[-patience:]
193
194         # If no improvement in recent epochs
195         if all(loss >= best_val_loss for loss in recent_val_losses):
196             return True
197
198         return False
199
200 def demonstrate_regularization():
201     """Demonstrate the effect of different regularization techniques
202     ."""
203
204     # Generate synthetic data
205     np.random.seed(42)
206     n_samples = 1000
207     n_features = 20
208
209     # True weights (sparse)
210     true_weights = np.zeros(n_features)
211     true_weights[:5] = [1.0, -0.5, 0.8, -1.2, 0.3] # Only first 5
212         are non-zero
213
214     # Generate data
215     X = np.random.randn(n_samples, n_features)
216     y = X @ true_weights + np.random.randn(n_samples) * 0.1
217
218     # Split data
219     train_size = int(0.8 * n_samples)
220     X_train, X_test = X[:train_size], X[train_size:]
221     y_train, y_test = y[:train_size], y[train_size:]
222
223     # Train linear regression with different regularization
224     def train_linear_regression(X, y, lambda_reg, reg_type='l2'):
225         """Train linear regression with regularization."""
226         n_features = X.shape[1]
227
228         # Closed-form solution with regularization
229         if reg_type == 'l2':
230             # Ridge regression:  $(X^T X + \lambda I)^{-1} X^T y$ 
231             A = X.T @ X + lambda_reg * np.eye(n_features)
232             weights = np.linalg.solve(A, X.T @ y)
233
234         elif reg_type == 'l1':
235             # LASSO - use coordinate descent (simplified)

```

```

234         from sklearn.linear_model import Lasso
235         model = Lasso(alpha=lambda_reg, max_iter=10000)
236         model.fit(X, y)
237         weights = model.coef_
238
239     return weights
240
241     # Test different regularization strengths
242     lambda_values = np.logspace(-4, 2, 20)
243     l2_weights_norms = []
244     l1_weights_norms = []
245     l2_test_errors = []
246     l1_test_errors = []
247
248     for lambda_reg in lambda_values:
249         # L2 regularization
250         weights_l2 = train_linear_regression(X_train, y_train,
251                                             lambda_reg, 'l2')
252         y_pred_l2 = X_test @ weights_l2
253         error_l2 = np.mean((y_test - y_pred_l2)**2)
254
255         l2_weights_norms.append(np.sum(weights_l2**2))
256         l2_test_errors.append(error_l2)
257
258         # L1 regularization
259         weights_l1 = train_linear_regression(X_train, y_train,
260                                             lambda_reg, 'l1')
261         y_pred_l1 = X_test @ weights_l1
262         error_l1 = np.mean((y_test - y_pred_l1)**2)
263
264         l1_weights_norms.append(np.sum(np.abs(weights_l1)))
265         l1_test_errors.append(error_l1)
266
267     # Plot results
268     fig, axes = plt.subplots(2, 2, figsize=(12, 10))
269
270     # L2 regularization: error vs lambda
271     axes[0, 0].semilogx(lambda_values, l2_test_errors, 'b-',
272                        linewidth=2)
273     axes[0, 0].set_xlabel('Regularization strength ($\lambda$)')
274     axes[0, 0].set_ylabel('Test MSE')
275     axes[0, 0].set_title('L2 Regularization: Error vs $\lambda$')
276     axes[0, 0].grid(True, alpha=0.3)
277
278     # L2 regularization: weight norm vs lambda
279     axes[0, 1].loglog(lambda_values, l2_weights_norms, 'r-',
280                      linewidth=2)

```

```

277 axes[0, 1].set_xlabel('Regularization strength ( $\lambda$ )')
278 axes[0, 1].set_ylabel(r'$||w||_2^2$')
279 axes[0, 1].set_title('L2: Weight Norm vs  $\lambda$ ')
280 axes[0, 1].grid(True, alpha=0.3)
281
282 # L1 regularization: error vs lambda
283 axes[1, 0].semilogx(lambda_values, l1_test_errors, 'g-',
284                      linewidth=2)
285 axes[1, 0].set_xlabel('Regularization strength ( $\lambda$ )')
286 axes[1, 0].set_ylabel('Test MSE')
287 axes[1, 0].set_title('L1 Regularization: Error vs  $\lambda$ ')
288 axes[1, 0].grid(True, alpha=0.3)
289
290 # L1 regularization: weight norm vs lambda
291 axes[1, 1].loglog(lambda_values, l1_weights_norms, 'orange',
292                   linewidth=2)
293 axes[1, 1].set_xlabel('Regularization strength ( $\lambda$ )')
294 axes[1, 1].set_ylabel('||w||_1')
295 axes[1, 1].set_title('L1: Weight Norm vs  $\lambda$ ')
296 axes[1, 1].grid(True, alpha=0.3)
297
298 plt.tight_layout()
299 plt.show()
300
301 # Show sparsity of L1 vs L2
302 lambda_opt = 0.1
303 weights_l2_opt = train_linear_regression(X_train, y_train,
304                                         lambda_opt, 'l2')
305 weights_l1_opt = train_linear_regression(X_train, y_train,
306                                         lambda_opt, 'l1')
307
308 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
309
310 axes[0].bar(range(n_features), weights_l2_opt, alpha=0.7)
311 axes[0].axhline(y=0, color='black', linestyle='--', alpha=0.3)
312 axes[0].set_xlabel('Feature index')
313 axes[0].set_ylabel('Weight value')
314 axes[0].set_title('L2 Regularization (Ridge)')
315 axes[0].grid(True, alpha=0.3)
316
317 axes[1].bar(range(n_features), weights_l1_opt, alpha=0.7, color=
318             'green')
319 axes[1].axhline(y=0, color='black', linestyle='--', alpha=0.3)
320 axes[1].set_xlabel('Feature index')
321 axes[1].set_ylabel('Weight value')
322 axes[1].set_title('L1 Regularization (LASSO) - Sparse Solution')
323 axes[1].grid(True, alpha=0.3)

```

```

319 plt.tight_layout()
320 plt.show()
321
322 # Count non-zero weights
323 n_nonzero_l2 = np.sum(np.abs(weights_l2_opt) > 1e-4)
324 n_nonzero_l1 = np.sum(np.abs(weights_l1_opt) > 1e-4)
325
326
327 print(f"Number of non-zero weights (threshold 1e-4):")
328 print(f"  L2 (Ridge): {n_nonzero_l2}/{n_features}")
329 print(f"  L1 (LASSO): {n_nonzero_l1}/{n_features}")
330 print(f"  True non-zero: 5/{n_features}")
331
332 return {
333     'l2_weights': weights_l2_opt,
334     'l1_weights': weights_l1_opt,
335     'true_weights': true_weights
336 }
337
338 # Demonstrate regularization
339 results = demonstrate_regularization()

```

Listing 4: Regularization Techniques Implementation

2.8 Weight Initialization

Why Initialization Matters: Poor initialization can lead to vanishing/exploding gradients.

Method	Initialization	Best For
Zeros	$W_{ij} = 0$	Never use!
Random	$W_{ij} \sim \mathcal{N}(0, 0.01)$	Simple networks
Xavier/Glorot	$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}\right)$	Sigmoid, Tanh
He	$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$	ReLU, Leaky ReLU
LeCun	$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{1}{n_{\text{in}}}}\right)$	SELU
Orthogonal	$W = Q$ where $QQ^T = I$	Recurrent networks

Table 2: Weight initialization methods for neural networks

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3

```

```

4 class WeightInitializer:
5     """Different weight initialization methods."""
6
7     @staticmethod
8     def zeros(shape):
9         """Initialize with zeros (bad practice, for comparison only)
10         ."""
11         return np.zeros(shape)
12
13     @staticmethod
14     def random_normal(shape, scale=0.01):
15         """Initialize with small random normal values."""
16         return np.random.randn(*shape) * scale
17
18     @staticmethod
19     def xavier(shape):
20         """
21         Xavier/Glorot initialization for sigmoid/tanh activations.
22
23         For uniform: U[-a, a] where a = sqrt(6/(fan_in + fan_out))
24         For normal: N(0, sqrt(2/(fan_in + fan_out)))
25         """
26         fan_in, fan_out = shape[0], shape[1]
27         scale = np.sqrt(2.0 / (fan_in + fan_out))
28         return np.random.randn(*shape) * scale
29
30     @staticmethod
31     def he(shape):
32         """
33         He initialization for ReLU activations.
34
35         For normal: N(0, sqrt(2/fan_in))
36         """
37         fan_in = shape[0]
38         scale = np.sqrt(2.0 / fan_in)
39         return np.random.randn(*shape) * scale
40
41     @staticmethod
42     def lecun(shape):
43         """
44         LeCun initialization for SELU activations.
45
46         For normal: N(0, sqrt(1/fan_in))
47         """
48         fan_in = shape[0]
49         scale = np.sqrt(1.0 / fan_in)
50         return np.random.randn(*shape) * scale

```

```

50
51 @staticmethod
52 def orthogonal(shape):
53     """
54     Orthogonal initialization.
55
56     Generates orthogonal matrix for square shapes,
57     or semi-orthogonal for rectangular shapes.
58     """
59     flat_shape = (shape[0], np.prod(shape[1:]))
60     a = np.random.randn(*flat_shape)
61     u, _, v = np.linalg.svd(a, full_matrices=False)
62     q = u if u.shape == flat_shape else v
63     q = q.reshape(shape)
64     return q
65
66
67 def analyze_initialization_effects():
68     """
69     Analyze the effect of different initialization methods
70     on forward and backward signal propagation.
71     """
72     np.random.seed(42)
73
74     # Network configuration
75     layer_sizes = [100, 50, 50, 10] # Input: 100, Hidden: 50, 50,
76                                     # Output: 10
77     n_layers = len(layer_sizes) - 1
78
79     # Different initialization methods to compare
80     init_methods = {
81         'Xavier': WeightInitializer.xavier,
82         'He': WeightInitializer.he,
83         'Small Random': lambda s: WeightInitializer.random_normal(s,
84                             scale=0.01),
85         'Large Random': lambda s: WeightInitializer.random_normal(s,
86                             scale=1.0)
87     }
88
89     n_samples = 1000 # Number of random inputs to test
90     results = {}
91
92     for method_name, init_func in init_methods.items():
93         print(f"\nAnalyzing {method_name} initialization...")
94
95         # Initialize network
96         weights = []

```

```

94     for l in range(n_layers):
95         shape = (layer_sizes[l+1], layer_sizes[l])
96         W = init_func(shape)
97         weights.append(W)
98
99     # Forward propagation analysis
100    activations = []
101    gradients = []
102
103    # Generate random input data
104    X = np.random.randn(layer_sizes[0], n_samples)
105
106    # Track statistics through forward pass
107    current_activation = X
108    activations.append(current_activation)
109
110    for l in range(n_layers):
111        # Linear transformation
112        z = weights[l] @ current_activation
113
114        # ReLU activation (most common)
115        current_activation = np.maximum(0, z)
116
117        activations.append(current_activation)
118
119    # Backward propagation analysis (simplified)
120    # Initialize gradient at output
121    dA = np.random.randn(*activations[-1].shape)
122
123    for l in reversed(range(n_layers)):
124        # Backprop through activation (ReLU derivative)
125        dZ = dA * (activations[l+1] > 0).astype(float)
126
127        # Backprop to previous layer
128        if l > 0:
129            dA = weights[l].T @ dZ
130
131        gradients.append(dZ)
132
133    # Reverse gradients list to match layer order
134    gradients = gradients[::-1]
135
136    # Compute statistics
137    activation_means = [np.mean(np.abs(a)) for a in activations]
138    activation_stds = [np.std(a) for a in activations]
139
140    gradient_means = [np.mean(np.abs(g)) for g in gradients]

```

```

141     gradient_stds = [np.std(g) for g in gradients]
142
143     results[method_name] = {
144         'activation_means': activation_means,
145         'activation_stds': activation_stds,
146         'gradient_means': gradient_means,
147         'gradient_stds': gradient_stds,
148         'weights': weights
149     }
150
151     print(f"    Activation means by layer: {[f'{m:.3f}' for m in
        activation_means]}")
152     print(f"    Activation stds by layer: {[f'{s:.3f}' for s in
        activation_stds]}")
153     print(f"    Gradient means by layer: {[f'{m:.3f}' for m in
        gradient_means]}")
154
155     # Plot results
156     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
157
158     # Activation means across layers
159     for method_name, result in results.items():
160         axes[0, 0].plot(result['activation_means'], 'o-', label=
            method_name, linewidth=2)
161
162     axes[0, 0].set_xlabel('Layer')
163     axes[0, 0].set_ylabel('Mean |Activation|')
164     axes[0, 0].set_title('Activation Means Across Layers')
165     axes[0, 0].legend()
166     axes[0, 0].grid(True, alpha=0.3)
167
168     # Activation stds across layers
169     for method_name, result in results.items():
170         axes[0, 1].plot(result['activation_stds'], 'o-', label=
            method_name, linewidth=2)
171
172     axes[0, 1].set_xlabel('Layer')
173     axes[0, 1].set_ylabel('Std of Activations')
174     axes[0, 1].set_title('Activation Standard Deviations')
175     axes[0, 1].legend()
176     axes[0, 1].grid(True, alpha=0.3)
177
178     # Gradient means across layers
179     for method_name, result in results.items():
180         axes[1, 0].plot(result['gradient_means'], 'o-', label=
            method_name, linewidth=2)
181

```

```

182 axes[1, 0].set_xlabel('Layer')
183 axes[1, 0].set_ylabel('Mean |Gradient|')
184 axes[1, 0].set_title('Gradient Means Across Layers (Backward)')
185 axes[1, 0].legend()
186 axes[1, 0].grid(True, alpha=0.3)
187
188 # Weight distributions histogram
189 ax = axes[1, 1]
190 for method_name, result in results.items():
191     # Flatten all weights
192     all_weights = np.concatenate([w.flatten() for w in result['
        weights']])
193
194     # Plot histogram (simplified to KDE)
195     from scipy.stats import gaussian_kde
196     if len(all_weights) > 1:
197         kde = gaussian_kde(all_weights)
198         x = np.linspace(min(all_weights), max(all_weights), 100)
199         ax.plot(x, kde(x), label=method_name, linewidth=2)
200
201 ax.set_xlabel('Weight Value')
202 ax.set_ylabel('Density')
203 ax.set_title('Weight Distributions')
204 ax.legend()
205 ax.grid(True, alpha=0.3)
206
207 plt.tight_layout()
208 plt.show()
209
210 # Check for vanishing/exploding gradients
211 print("\n" + "="*60)
212 print("Vanishing/Exploding Gradient Analysis:")
213 print("="*60)
214
215 for method_name, result in results.items():
216     grad_means = result['gradient_means']
217
218     # Check if gradients are vanishing
219     vanishing_ratio = grad_means[-1] / grad_means[0] if
        grad_means[0] > 0 else 0
220     exploding = any(m > 100 for m in grad_means)
221     vanishing = vanishing_ratio < 0.01
222
223     status = " Good"
224     if exploding:
225         status = "Exploding"
226     elif vanishing:

```

```

227         status = " Vanishing"
228
229         print(f"{method_name:15} Gradient ratio (last/first): {
           vanishing_ratio:.6f} {status}")
230
231     return results
232
233 # Run initialization analysis
234 results = analyze_initialization_effects()

```

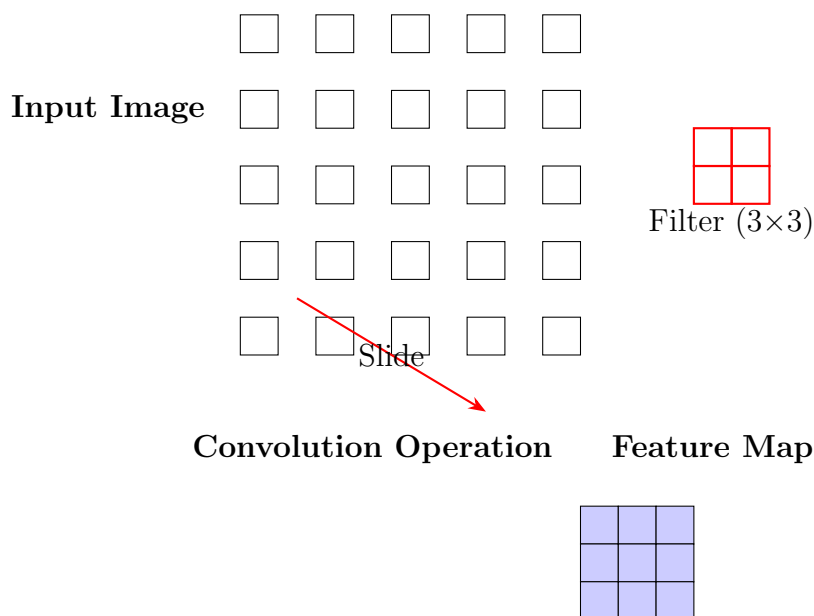
Listing 5: Weight Initialization Comparison

3 Convolutional Neural Networks (CNNs)

3.1 Introduction to CNNs

The Problem with Fully Connected Networks: For images (e.g., $224 \times 224 \times 3 = 150,528$ pixels), fully connected networks would have billions of parameters and ignore spatial structure.

CNN Insight: Instead of connecting every neuron to every input pixel, use convolution operations that preserve spatial relationships and share weights across the image.



Convolution Operation: $(I * K)[i, j] = \sum_m \sum_n I[i + m, j + n] \cdot K[m, n]$
 where I is input image, K is kernel/filter

Figure 8: Convolution operation: Filter slides across image, computing dot products to produce feature map

3.2 CNN Components

3.2.1 Convolutional Layer

Key Properties:

- **Local Connectivity:** Neurons connect only to local region of input
- **Weight Sharing:** Same filter used across entire image
- **Translation Equivariance:** Pattern detection regardless of position

Definition 3.1 (2D Convolution). *For input I of size $H \times W$ and kernel K of size $k_h \times k_w$:*

$$(I * K)[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I[i + m, j + n] \cdot K[m, n]$$

3.2.2 Pooling Layer

Purpose: Reduce spatial dimensions, increase translation invariance.

- **Max Pooling:** Take maximum value in window
- **Average Pooling:** Take average value in window

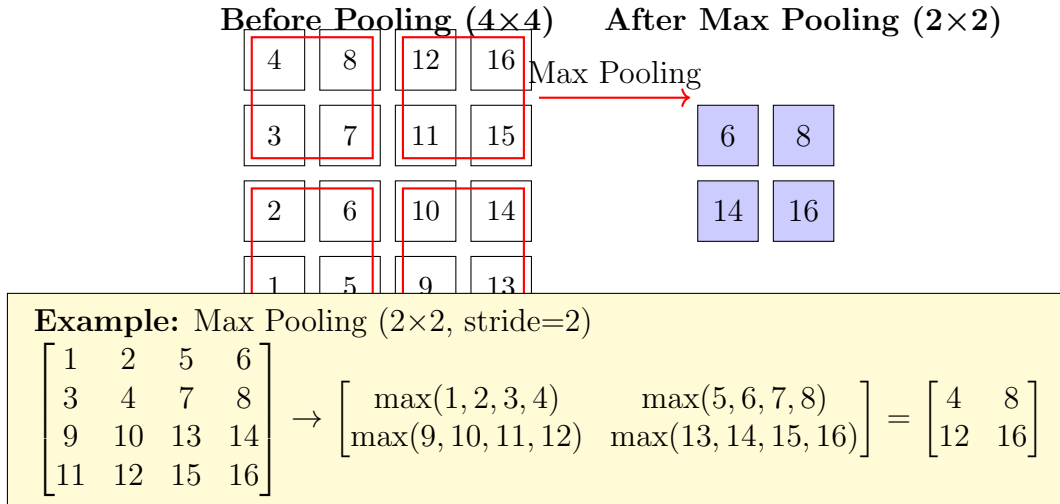


Figure 9: Max Pooling: Reduces spatial dimensions while preserving important features

3.2.3 Fully Connected Layer

After convolution and pooling, flattened features are passed to fully connected layers for classification.

3.3 CNN Architectures

3.3.1 LeNet-5 (1998)

First successful CNN for digit recognition.

- Input: 32×32 grayscale images
- Architecture: Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC $\rightarrow$ FC $\rightarrow$ Output
- Key insight: Use convolution for feature extraction

3.3.2 AlexNet (2012)

Revolutionized computer vision (ImageNet competition).

- Deeper network (8 layers)
- ReLU activation instead of sigmoid
- Dropout for regularization
- GPU implementation

3.3.3 VGGNet (2014)

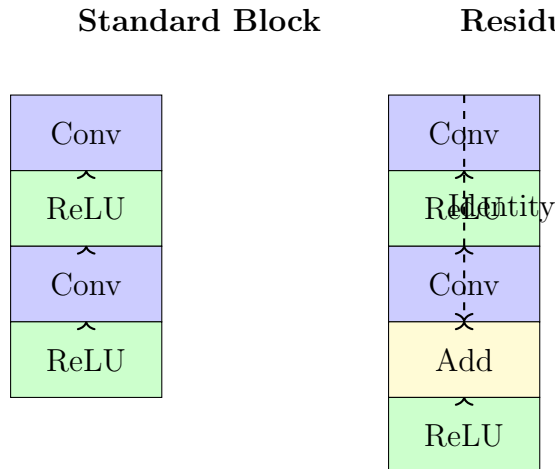
Very deep network with simple architecture.

- Only 3×3 convolutions, 2×2 pooling
- VGG-16: 16 weight layers
- VGG-19: 19 weight layers

3.3.4 ResNet (2015)

Solved vanishing gradient problem with residual connections.

- Residual blocks: $H(x) = F(x) + x$
- Can train very deep networks (152+ layers)
- Won ImageNet 2015



Residual Learning: Instead of learning $H(x)$, learn residual $F(x) = H(x) - x$, then $H(x) = F(x) + x$

Figure 10: Residual blocks enable training of very deep networks by learning residuals

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from typing import List, Tuple
4
5 class Conv2D:
6     """2D Convolution layer."""
7
8     def __init__(self, in_channels: int, out_channels: int,
9                 kernel_size: int,
10                 stride: int = 1, padding: int = 0, bias: bool =
11                 True):
12
13         """
14         Initialize 2D convolution layer.
15
16         Args:
17             in_channels: Number of input channels
18             out_channels: Number of output channels (number of
19                         filters)
20             kernel_size: Size of convolution kernel (assumed square)
21             stride: Stride of convolution
22             padding: Padding to apply to input
23             bias: Whether to use bias term
24
25         """
26         self.in_channels = in_channels
27         self.out_channels = out_channels
28         self.kernel_size = kernel_size
29         self.stride = stride

```

```

25     self.padding = padding
26     self.use_bias = bias
27
28     # Initialize parameters
29     # He initialization for ReLU
30     scale = np.sqrt(2.0 / (in_channels * kernel_size *
31                             kernel_size))
32     self.weights = np.random.randn(out_channels, in_channels,
33                                     kernel_size, kernel_size) *
34                                     scale
35
36     if bias:
37         self.bias = np.zeros(out_channels)
38     else:
39         self.bias = None
40
41     # Cache for backward pass
42     self.cache = None
43
44 def forward(self, X: np.ndarray) -> np.ndarray:
45     """
46     Forward pass for 2D convolution.
47
48     Args:
49         X: Input of shape (batch_size, in_channels, height,
50                             width)
51
52     Returns:
53         Output of shape (batch_size, out_channels, out_height,
54                             out_width)
55     """
56     batch_size, in_channels, in_height, in_width = X.shape
57
58     # Add padding if needed
59     if self.padding > 0:
60         X_padded = np.pad(X, ((0, 0), (0, 0),
61                                (self.padding, self.padding),
62                                (self.padding, self.padding)),
63                             mode='constant')
64     else:
65         X_padded = X
66
67     # Compute output dimensions
68     out_height = (in_height + 2*self.padding - self.kernel_size)
69                 // self.stride + 1
70     out_width = (in_width + 2*self.padding - self.kernel_size)
71                // self.stride + 1

```

```

66
67     # Initialize output
68     output = np.zeros((batch_size, self.out_channels, out_height
69                        , out_width))
69
70     # Perform convolution
71     for b in range(batch_size):
72         for c_out in range(self.out_channels):
73             for h_out in range(out_height):
74                 for w_out in range(out_width):
75                     # Extract patch
76                     h_start = h_out * self.stride
77                     h_end = h_start + self.kernel_size
78                     w_start = w_out * self.stride
79                     w_end = w_start + self.kernel_size
80
81                     patch = X_padded[b, :, h_start:h_end,
82                                     w_start:w_end]
83
84                     # Compute convolution
85                     output[b, c_out, h_out, w_out] = np.sum(
86                         patch * self.weights[c_out]
87                     )
88
89                     # Add bias if used
90                     if self.use_bias:
91                         output[b, c_out] += self.bias[c_out]
92
93     # Cache for backward pass
94     self.cache = (X, X_padded)
95
96     return output
97
98 def backward(self, dout: np.ndarray, learning_rate: float =
99             0.01) -> np.ndarray:
100     """
101     Backward pass for 2D convolution.
102
103     Args:
104         dout: Gradient from next layer
105         learning_rate: Learning rate for parameter updates
106
107     Returns:
108         Gradient for input
109     """
110     X, X_padded = self.cache
111     batch_size, in_channels, in_height, in_width = X.shape

```

```

110     _, out_channels, out_height, out_width = dout.shape
111
112     # Initialize gradients
113     dX_padded = np.zeros_like(X_padded)
114     dW = np.zeros_like(self.weights)
115     db = np.zeros_like(self.bias) if self.use_bias else None
116
117     # Compute gradients
118     for b in range(batch_size):
119         for c_out in range(out_channels):
120             if self.use_bias:
121                 db[c_out] += np.sum(dout[b, c_out])
122
123         for h_out in range(out_height):
124             for w_out in range(out_width):
125                 # Extract patch position
126                 h_start = h_out * self.stride
127                 h_end = h_start + self.kernel_size
128                 w_start = w_out * self.stride
129                 w_end = w_start + self.kernel_size
130
131                 # Gradient for weights
132                 patch = X_padded[b, :, h_start:h_end,
133                                     w_start:w_end]
134                 dW[c_out] += dout[b, c_out, h_out, w_out] *
135                             patch
136
137                 # Gradient for input
138                 dX_padded[b, :, h_start:h_end, w_start:w_end
139                             ] += \
140                     dout[b, c_out, h_out, w_out] * self.
141                         weights[c_out]
142
143     # Remove padding from input gradient
144     if self.padding > 0:
145         dX = dX_padded[:, :, self.padding:-self.padding, self.
146                         padding:-self.padding]
147     else:
148         dX = dX_padded
149
150     # Update parameters
151     self.weights -= learning_rate * dW / batch_size
152     if self.use_bias:
153         self.bias -= learning_rate * db / batch_size
154
155     return dX

```

```

152
153 class MaxPool2D:
154     """2D Max Pooling layer."""
155
156     def __init__(self, pool_size: int = 2, stride: int = 2):
157         """
158         Initialize 2D max pooling layer.
159
160         Args:
161             pool_size: Size of pooling window (assumed square)
162             stride: Stride of pooling
163         """
164         self.pool_size = pool_size
165         self.stride = stride
166         self.cache = None
167
168     def forward(self, X: np.ndarray) -> np.ndarray:
169         """
170         Forward pass for max pooling.
171
172         Args:
173             X: Input of shape (batch_size, channels, height, width)
174
175         Returns:
176             Output of shape (batch_size, channels, out_height,
177                             out_width)
178         """
179         batch_size, channels, in_height, in_width = X.shape
180
181         # Compute output dimensions
182         out_height = (in_height - self.pool_size) // self.stride + 1
183         out_width = (in_width - self.pool_size) // self.stride + 1
184
185         # Initialize output
186         output = np.zeros((batch_size, channels, out_height,
187                             out_width))
188
189         # Cache for backward pass (store max positions)
190         max_positions = np.zeros((batch_size, channels, out_height,
191                                 out_width, 2), dtype=int)
192
193         # Perform max pooling
194         for b in range(batch_size):
195             for c in range(channels):
196                 for h_out in range(out_height):
197                     for w_out in range(out_width):
198                         h_start = h_out * self.stride

```

```

196         h_end = h_start + self.pool_size
197         w_start = w_out * self.stride
198         w_end = w_start + self.pool_size
199
200         patch = X[b, c, h_start:h_end, w_start:w_end
201                 ]
202         output[b, c, h_out, w_out] = np.max(patch)
203
204         # Store position of max for backward pass
205         max_pos = np.unravel_index(np.argmax(patch),
206                                   patch.shape)
207         max_positions[b, c, h_out, w_out] = [
208             h_start + max_pos[0],
209             w_start + max_pos[1]
210         ]
211
212     self.cache = (X.shape, max_positions)
213     return output
214
215 def backward(self, dout: np.ndarray) -> np.ndarray:
216     """
217     Backward pass for max pooling.
218
219     Args:
220         dout: Gradient from next layer
221
222     Returns:
223         Gradient for input
224     """
225     X_shape, max_positions = self.cache
226     batch_size, channels, in_height, in_width = X_shape
227     _, _, out_height, out_width = dout.shape
228
229     # Initialize gradient for input
230     dX = np.zeros(X_shape)
231
232     # Distribute gradient to max positions
233     for b in range(batch_size):
234         for c in range(channels):
235             for h_out in range(out_height):
236                 for w_out in range(out_width):
237                     h_max, w_max = max_positions[b, c, h_out,
238                                                 w_out]
239                     dX[b, c, h_max, w_max] += dout[b, c, h_out,
240                                                     w_out]
241
242     return dX

```

```

239
240
241 class Flatten:
242     """Flatten layer for CNN to FC transition."""
243
244     def __init__(self):
245         self.cache = None
246
247     def forward(self, X: np.ndarray) -> np.ndarray:
248         """
249         Forward pass for flattening.
250
251         Args:
252             X: Input of shape (batch_size, channels, height, width)
253
254         Returns:
255             Flattened output of shape (batch_size, channels*height*
256                 width)
257         """
258         self.cache = X.shape
259         batch_size = X.shape[0]
260         return X.reshape(batch_size, -1)
261
262     def backward(self, dout: np.ndarray) -> np.ndarray:
263         """
264         Backward pass for flattening.
265
266         Args:
267             dout: Gradient from next layer
268
269         Returns:
270             Gradient reshaped to original input shape
271         """
272         original_shape = self.cache
273         return dout.reshape(original_shape)
274
275 class CNN:
276     """Simple CNN model for image classification."""
277
278     def __init__(self, input_shape: Tuple[int, int, int],
279                 num_classes: int):
280         """
281         Initialize CNN model.
282
283         Args:
284             input_shape: (channels, height, width)

```

```

284         num_classes: Number of output classes
285     """
286     self.layers = []
287
288     # Build CNN architecture (similar to LeNet)
289     # Conv1: 1x28x28 -> 6x24x24
290     self.layers.append(Conv2D(
291         in_channels=input_shape[0],
292         out_channels=6,
293         kernel_size=5,
294         stride=1,
295         padding=0
296     ))
297
298     # ReLU activation
299     self.layers.append(lambda x: np.maximum(0, x)) # Simplified
        ReLU
300
301     # MaxPool1: 6x24x24 -> 6x12x12
302     self.layers.append(MaxPool2D(pool_size=2, stride=2))
303
304     # Conv2: 6x12x12 -> 16x8x8
305     self.layers.append(Conv2D(
306         in_channels=6,
307         out_channels=16,
308         kernel_size=5,
309         stride=1,
310         padding=0
311     ))
312
313     # ReLU activation
314     self.layers.append(lambda x: np.maximum(0, x))
315
316     # MaxPool2: 16x8x8 -> 16x4x4
317     self.layers.append(MaxPool2D(pool_size=2, stride=2))
318
319     # Flatten: 16x4x4 -> 256
320     self.layers.append(Flatten())
321
322     # Fully connected layers (simplified as linear layers)
323     # FC1: 256 -> 120
324     self.layers.append(self._create_fc_layer(16*4*4, 120))
325
326     # ReLU
327     self.layers.append(lambda x: np.maximum(0, x))
328
329     # FC2: 120 -> 84

```

```

330     self.layers.append(self._create_fc_layer(120, 84))
331
332     # ReLU
333     self.layers.append(lambda x: np.maximum(0, x))
334
335     # Output: 84 -> num_classes
336     self.layers.append(self._create_fc_layer(84, num_classes))
337
338     # Store which layers have parameters
339     self.param_layers = [0, 3, 7, 9, 11] # Indices of layers
        with parameters
340
341     def _create_fc_layer(self, input_dim: int, output_dim: int):
342         """Create a fully connected layer (simplified)."""
343         # He initialization
344         scale = np.sqrt(2.0 / input_dim)
345         W = np.random.randn(output_dim, input_dim) * scale
346         b = np.zeros(output_dim)
347
348         return {'W': W, 'b': b, 'input_dim': input_dim}
349
350     def forward(self, X: np.ndarray) -> np.ndarray:
351         """
352         Forward pass through CNN.
353
354         Args:
355             X: Input of shape (batch_size, channels, height, width)
356
357         Returns:
358             Output logits
359         """
360         activations = [X]
361
362         for layer in self.layers:
363             if callable(layer):
364                 # Activation function or simple layer
365                 output = layer(activations[-1])
366             elif isinstance(layer, dict):
367                 # Fully connected layer
368                 W, b = layer['W'], layer['b']
369                 # Reshape if needed
370                 if activations[-1].ndim > 2:
371                     batch_size = activations[-1].shape[0]
372                     input_flat = activations[-1].reshape(batch_size,
373                                                             -1)
374                 else:
375                     input_flat = activations[-1]

```

```

375         output = input_flat @ W.T + b
376         layer['input'] = input_flat # Cache for backward
377         pass
378     else:
379         # Conv2D, MaxPool2D, or Flatten layer
380         output = layer.forward(activations[-1])
381
382     activations.append(output)
383
384     return activations[-1]
385
386 def backward(self, dout: np.ndarray, learning_rate: float =
387 0.01):
388     """
389     Backward pass through CNN.
390
391     Args:
392         dout: Gradient of loss w.r.t. output
393         learning_rate: Learning rate
394     """
395     # Backpropagate through layers in reverse order
396     current_grad = dout
397
398     for i in reversed(range(len(self.layers))):
399         layer = self.layers[i]
400
401         if i in self.param_layers:
402             # Parameter layer (Conv2D or FC)
403             if isinstance(layer, dict):
404                 # Fully connected layer
405                 W, b = layer['W'], layer['b']
406                 input_flat = layer['input']
407
408                 # Gradient for weights
409                 dW = current_grad.T @ input_flat / current_grad.
410                     shape[0]
411                 db = np.mean(current_grad, axis=0)
412
413                 # Gradient for input
414                 if i > 0:
415                     dx = current_grad @ W
416
417                 # Reshape if next layer expects different
418                 shape
419                 prev_layer = self.layers[i-1]
420                 if isinstance(prev_layer, (Conv2D, MaxPool2D

```

```

418         , Flatten)):
419             # Need to reshape to match previous
420             # layer's output
421             # For simplicity, we'll skip this in
422             # this example
423             pass
424
425             current_grad = dx
426         else:
427             current_grad = None
428
429             # Update parameters
430             layer['W'] -= learning_rate * dW
431             layer['b'] -= learning_rate * db
432
433         elif hasattr(layer, 'backward'):
434             # Conv2D layer
435             current_grad = layer.backward(current_grad,
436                                           learning_rate)
437
438         elif hasattr(layer, 'backward'):
439             # MaxPool2D or Flatten layer
440             current_grad = layer.backward(current_grad)
441
442         elif callable(layer):
443             # Activation function (ReLU)
444             # For ReLU: gradient is 1 where input > 0, else 0
445             # In our forward pass, we need to track inputs to
446             # activations
447             # For simplicity, we'll skip this in this example
448             pass
449
450 def visualize_cnn_features():
451     """Visualize what different CNN filters learn."""
452     np.random.seed(42)
453
454     # Create a simple CNN
455     cnn = CNN(input_shape=(1, 28, 28), num_classes=10)
456
457     # Visualize first convolutional layer filters
458     conv1 = cnn.layers[0] # First Conv2D layer
459     filters = conv1.weights # Shape: (6, 1, 5, 5)
460
461     fig, axes = plt.subplots(2, 3, figsize=(10, 6))
462     axes = axes.flatten()

```

```

460     for i in range(6):
461         filter_img = filters[i, 0] # First channel (grayscale input
462         )
463         im = axes[i].imshow(filter_img, cmap='viridis')
464         axes[i].set_title(f'Filter {i+1}')
465         axes[i].axis('off')
466         plt.colorbar(im, ax=axes[i])
467
468     plt.suptitle('First Convolutional Layer Filters', fontsize=16)
469     plt.tight_layout()
470     plt.show()
471
472     # Apply filters to sample image
473     from scipy import misc
474
475     # Load sample image (or create synthetic)
476     face = misc.face(gray=True)
477     face = misc.imresize(face, (28, 28)) / 255.0
478
479     # Add batch and channel dimensions
480     X = face.reshape(1, 1, 28, 28)
481
482     # Apply first conv layer
483     conv_output = conv1.forward(X) # Shape: (1, 6, 24, 24)
484
485     # Visualize feature maps
486     fig, axes = plt.subplots(2, 4, figsize=(12, 6))
487
488     # Original image
489     axes[0, 0].imshow(face, cmap='gray')
490     axes[0, 0].set_title('Original Image')
491     axes[0, 0].axis('off')
492
493     # Feature maps
494     for i in range(6):
495         row, col = divmod(i+1, 4)
496         feature_map = conv_output[0, i]
497         im = axes[row, col].imshow(feature_map, cmap='hot')
498         axes[row, col].set_title(f'Feature Map {i+1}')
499         axes[row, col].axis('off')
500         plt.colorbar(im, ax=axes[row, col])
501
502     plt.suptitle('Feature Maps from First Convolutional Layer',
503                 fontsize=16)
504     plt.tight_layout()
505     plt.show()

```

```

505     return cnn, face, conv_output
506
507 # Visualize CNN features
508 cnn, sample_image, features = visualize_cnn_features()

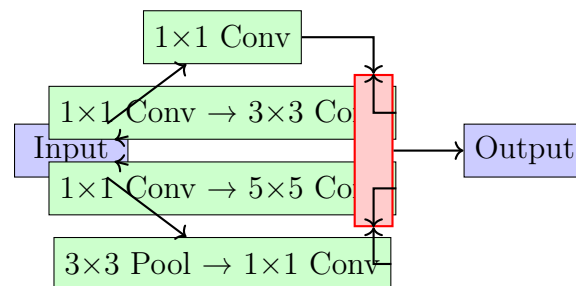
```

Listing 6: CNN Implementation from Scratch

3.4 Advanced CNN Architectures

3.4.1 Inception Networks

Idea: Instead of choosing filter size, use multiple filter sizes in parallel.



1×1 Convolution as Bottleneck: Reduces computational cost
 Example: 192 channels → 1×1 conv (64 filters) → 3×3 conv (128 filters)
 Operations: Without bottleneck: $192 \times 3 \times 128 = 221,184$
 With bottleneck: $192 \times 1 \times 64 + 64 \times 3 \times 128 = 12,288 + 73,728 = 86,016$ ($2.6\times$ reduction)

Figure 11: Inception module: Multiple filter sizes processed in parallel then concatenated

3.4.2 DenseNet

Idea: Each layer receives feature maps from all preceding layers.

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

where $[\cdot]$ denotes concatenation.

3.4.3 EfficientNet

Idea: Compound scaling of depth, width, and resolution using neural architecture search.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class InceptionModule(nn.Module):
6     """Inception module from GoogLeNet."""
7

```

```

8   def __init__(self, in_channels, out_1x1, red_3x3, out_3x3,
9       red_5x5, out_5x5, out_pool):
10       super(InceptionModule, self).__init__()
11
12       # 1x1 convolution branch
13       self.branch1 = nn.Sequential(
14           nn.Conv2d(in_channels, out_1x1, kernel_size=1),
15           nn.BatchNorm2d(out_1x1),
16           nn.ReLU(inplace=True)
17       )
18
19       # 1x1 -> 3x3 convolution branch
20       self.branch2 = nn.Sequential(
21           nn.Conv2d(in_channels, red_3x3, kernel_size=1),
22           nn.BatchNorm2d(red_3x3),
23           nn.ReLU(inplace=True),
24           nn.Conv2d(red_3x3, out_3x3, kernel_size=3, padding=1),
25           nn.BatchNorm2d(out_3x3),
26           nn.ReLU(inplace=True)
27       )
28
29       # 1x1 -> 5x5 convolution branch
30       self.branch3 = nn.Sequential(
31           nn.Conv2d(in_channels, red_5x5, kernel_size=1),
32           nn.BatchNorm2d(red_5x5),
33           nn.ReLU(inplace=True),
34           nn.Conv2d(red_5x5, out_5x5, kernel_size=5, padding=2),
35           nn.BatchNorm2d(out_5x5),
36           nn.ReLU(inplace=True)
37       )
38
39       # 3x3 pool -> 1x1 convolution branch
40       self.branch4 = nn.Sequential(
41           nn.MaxPool2d(kernel_size=3, stride=1, padding=1),
42           nn.Conv2d(in_channels, out_pool, kernel_size=1),
43           nn.BatchNorm2d(out_pool),
44           nn.ReLU(inplace=True)
45       )
46
47       def forward(self, x):
48           branch1 = self.branch1(x)
49           branch2 = self.branch2(x)
50           branch3 = self.branch3(x)
51           branch4 = self.branch4(x)
52
53       # Concatenate along channel dimension
54       output = torch.cat([branch1, branch2, branch3, branch4], dim

```

```

        =1)
54         return output
55
56
57 class ResidualBlock(nn.Module):
58     """Basic residual block from ResNet."""
59
60     def __init__(self, in_channels, out_channels, stride=1,
61                 downsample=None):
62         super(ResidualBlock, self).__init__()
63
64         self.conv1 = nn.Conv2d(in_channels, out_channels,
65                                 kernel_size=3,
66                                 stride=stride, padding=1, bias=False)
67         self.bn1 = nn.BatchNorm2d(out_channels)
68         self.relu = nn.ReLU(inplace=True)
69         self.conv2 = nn.Conv2d(out_channels, out_channels,
70                                 kernel_size=3,
71                                 stride=1, padding=1, bias=False)
72         self.bn2 = nn.BatchNorm2d(out_channels)
73         self.downsample = downsample
74
75     def forward(self, x):
76         identity = x
77
78         out = self.conv1(x)
79         out = self.bn1(out)
80         out = self.relu(out)
81
82         out = self.conv2(out)
83         out = self.bn2(out)
84
85         if self.downsample is not None:
86             identity = self.downsample(x)
87
88         out += identity
89         out = self.relu(out)
90
91         return out
92
93
94 class DenseLayer(nn.Module):
95     """Dense layer from DenseNet."""
96
97     def __init__(self, in_channels, growth_rate):
98         super(DenseLayer, self).__init__()

```

```

97         self.bn1 = nn.BatchNorm2d(in_channels)
98         self.relu = nn.ReLU(inplace=True)
99         self.conv1 = nn.Conv2d(in_channels, 4 * growth_rate,
100                                kernel_size=1, bias=False)
101
102         self.bn2 = nn.BatchNorm2d(4 * growth_rate)
103         self.conv2 = nn.Conv2d(4 * growth_rate, growth_rate,
104                                kernel_size=3,
105                                padding=1, bias=False)
106
107     def forward(self, x):
108         out = self.bn1(x)
109         out = self.relu(out)
110         out = self.conv1(out)
111
112         out = self.bn2(out)
113         out = self.relu(out)
114         out = self.conv2(out)
115
116         return out
117
118 class DenseBlock(nn.Module):
119     """Dense block from DenseNet."""
120
121     def __init__(self, in_channels, num_layers, growth_rate):
122         super(DenseBlock, self).__init__()
123
124         self.layers = nn.ModuleList()
125
126         for i in range(num_layers):
127             layer = DenseLayer(in_channels + i * growth_rate,
128                                growth_rate)
129             self.layers.append(layer)
130
131     def forward(self, x):
132         features = [x]
133
134         for layer in self.layers:
135             new_features = layer(torch.cat(features, dim=1))
136             features.append(new_features)
137
138         return torch.cat(features, dim=1)
139
140 class SqueezeExcitation(nn.Module):
141     """Squeeze-and-Excitation block from SENet."""

```

```

141
142     def __init__(self, in_channels, reduction_ratio=16):
143         super(SqueezeExcitation, self).__init__()
144
145         reduced_channels = max(1, in_channels // reduction_ratio)
146
147         self.squeeze = nn.AdaptiveAvgPool2d(1)
148         self.excitation = nn.Sequential(
149             nn.Linear(in_channels, reduced_channels, bias=False),
150             nn.ReLU(inplace=True),
151             nn.Linear(reduced_channels, in_channels, bias=False),
152             nn.Sigmoid()
153         )
154
155     def forward(self, x):
156         batch_size, channels, _, _ = x.size()
157
158         # Squeeze: global average pooling
159         squeezed = self.squeeze(x).view(batch_size, channels)
160
161         # Excitation: learn channel-wise weights
162         weights = self.excitation(squeezed).view(batch_size,
163             channels, 1, 1)
164
165         # Scale: apply channel weights
166         return x * weights.expand_as(x)
167
168 class MBConvBlock(nn.Module):
169     """Mobile Inverted Bottleneck Conv Block from EfficientNet."""
170
171     def __init__(self, in_channels, out_channels, expansion_ratio,
172         stride,
173             kernel_size=3, se_ratio=0.25):
174         super(MBConvBlock, self).__init__()
175
176         expanded_channels = in_channels * expansion_ratio
177         use_se = (se_ratio is not None) and (0 < se_ratio <= 1)
178
179         # Expansion phase (if expansion_ratio != 1)
180         if expansion_ratio != 1:
181             self.expand_conv = nn.Conv2d(in_channels,
182                 expanded_channels,
183                     kernel_size=1, bias=False)
184             self.bn0 = nn.BatchNorm2d(expanded_channels)
185             self.swish0 = nn.SiLU() # Swish activation

```

```

185     # Depthwise convolution
186     self.depthwise_conv = nn.Conv2d(
187         expanded_channels, expanded_channels, kernel_size=
            kernel_size,
188         stride=stride, padding=kernel_size//2, groups=
            expanded_channels, bias=False
189     )
190     self.bn1 = nn.BatchNorm2d(expanded_channels)
191     self.swish1 = nn.SiLU()
192
193     # Squeeze-and-Excitation
194     if use_se:
195         reduced_channels = max(1, int(expanded_channels *
            se_ratio))
196         self.se = SqueezeExcitation(expanded_channels,
197                                     reduction_ratio=
            expanded_channels//
            reduced_channels)
198
199     # Output phase
200     self.project_conv = nn.Conv2d(expanded_channels,
            out_channels,
201                                     kernel_size=1, bias=False)
202     self.bn2 = nn.BatchNorm2d(out_channels)
203
204     # Skip connection if input and output have same dimensions
205     self.use_skip = (stride == 1) and (in_channels ==
            out_channels)
206
207     def forward(self, x):
208         identity = x
209
210         # Expansion
211         if hasattr(self, 'expand_conv'):
212             x = self.expand_conv(x)
213             x = self.bn0(x)
214             x = self.swish0(x)
215
216         # Depthwise convolution
217         x = self.depthwise_conv(x)
218         x = self.bn1(x)
219         x = self.swish1(x)
220
221         # Squeeze-and-Excitation
222         if hasattr(self, 'se'):
223             x = self.se(x)
224

```

```

225         # Projection
226         x = self.project_conv(x)
227         x = self.bn2(x)
228
229         # Skip connection
230         if self.use_skip:
231             x = x + identity
232
233         return x
234
235
236 def create_resnet18():
237     """Create ResNet-18 model."""
238     class ResNet18(nn.Module):
239         def __init__(self, num_classes=10):
240             super(ResNet18, self).__init__()
241
242             self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2,
243                                     padding=3, bias=False)
244             self.bn1 = nn.BatchNorm2d(64)
245             self.relu = nn.ReLU(inplace=True)
246             self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2,
247                                           padding=1)
248
249             # Residual blocks
250             self.layer1 = self._make_layer(64, 64, 2, stride=1)
251             self.layer2 = self._make_layer(64, 128, 2, stride=2)
252             self.layer3 = self._make_layer(128, 256, 2, stride=2)
253             self.layer4 = self._make_layer(256, 512, 2, stride=2)
254
255             self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
256             self.fc = nn.Linear(512, num_classes)
257
258         def _make_layer(self, in_channels, out_channels, blocks,
259                        stride):
260             downsample = None
261             if stride != 1 or in_channels != out_channels:
262                 downsample = nn.Sequential(
263                     nn.Conv2d(in_channels, out_channels, kernel_size
264                               =1,
265                               stride=stride, bias=False),
266                     nn.BatchNorm2d(out_channels)
267                 )
268
269             layers = []
270             layers.append(ResidualBlock(in_channels, out_channels,
271                                         stride, downsample))

```

```

267         for _ in range(1, blocks):
268             layers.append(ResidualBlock(out_channels,
269                                         out_channels))
270
271         return nn.Sequential(*layers)
272
273     def forward(self, x):
274         x = self.conv1(x)
275         x = self.bn1(x)
276         x = self.relu(x)
277         x = self.maxpool(x)
278
279         x = self.layer1(x)
280         x = self.layer2(x)
281         x = self.layer3(x)
282         x = self.layer4(x)
283
284         x = self.avgpool(x)
285         x = torch.flatten(x, 1)
286         x = self.fc(x)
287
288         return x
289
290     return ResNet18()
291
292
293 def print_model_summary(model, input_size=(1, 3, 224, 224)):
294     """Print model summary with parameter count."""
295     from torchsummary import summary
296     summary(model, input_size[1:])
297
298     # Calculate total parameters
299     total_params = sum(p.numel() for p in model.parameters())
300     trainable_params = sum(p.numel() for p in model.parameters() if
301                             p.requires_grad)
302
303     print(f"\nTotal parameters: {total_params:,}")
304     print(f"Trainable parameters: {trainable_params:,}")
305     print(f"Non-trainable parameters: {total_params -
306                                         trainable_params:,}")
307
308 # Create and compare different CNN architectures
309 print("="*60)
310 print("ResNet-18 Architecture")
311 print("="*60)

```

```

311 resnet18 = create_resnet18()
312 print_model_summary(resnet18, input_size=(1, 3, 224, 224))
313
314 # Compare model sizes and FLOPs
315 def calculate_flops(model, input_size=(1, 3, 224, 224)):
316     """Calculate FLOPs for model (simplified)."""
317     # This is a simplified calculation
318     # In practice, use tools like torchprofile or fvcore
319     total_flops = 0
320
321     for name, module in model.named_modules():
322         if isinstance(module, nn.Conv2d):
323             # Conv2D FLOPs = output_h * output_w * in_c * out_c *
324                 kernel_h * kernel_w
325             # Simplified calculation
326             output_size = input_size[2] // module.stride[0] #
327                 Approximate
328             kernel_ops = module.kernel_size[0] * module.kernel_size
329                 [1]
330             flops = (output_size ** 2) * module.in_channels * module
331                 .out_channels * kernel_ops
332             total_flops += flops
333
334         elif isinstance(module, nn.Linear):
335             # Linear FLOPs = in_features * out_features
336             flops = module.in_features * module.out_features
337             total_flops += flops
338
339     return total_flops
340
341 print(f"\nApproximate FLOPs for ResNet-18: {calculate_flops(resnet18
342 ):,}")
343
344 # Test different blocks
345 print("\n" + "="*60)
346 print("Testing Different CNN Blocks")
347 print("="*60)
348
349 # Test Inception module
350 inception = InceptionModule(in_channels=192, out_1x1=64, red_3x3=96,
351                             out_3x3=128, red_5x5=16, out_5x5=32,
352                             out_pool=32)
353 test_input = torch.randn(1, 192, 28, 28)
354 output = inception(test_input)
355 print(f"Inception module: Input {tuple(test_input.shape)} -> Output
356       {tuple(output.shape)}")

```

```

351 # Test Residual block
352 residual = ResidualBlock(in_channels=64, out_channels=64)
353 test_input = torch.randn(1, 64, 56, 56)
354 output = residual(test_input)
355 print(f"Residual block: Input {tuple(test_input.shape)} -> Output {
      tuple(output.shape)}")
356
357 # Test Dense block
358 dense = DenseBlock(in_channels=64, num_layers=4, growth_rate=32)
359 test_input = torch.randn(1, 64, 28, 28)
360 output = dense(test_input)
361 print(f"Dense block (4 layers, growth=32): Input {tuple(test_input.
      shape)} -> Output {tuple(output.shape)}")
362
363 # Test MBConv block
364 mbconv = MBConvBlock(in_channels=32, out_channels=16,
      expansion_ratio=6, stride=1)
365 test_input = torch.randn(1, 32, 28, 28)
366 output = mbconv(test_input)
367 print(f"MBConv block: Input {tuple(test_input.shape)} -> Output {
      tuple(output.shape)}")

```

Listing 7: Advanced CNN Architectures Implementation

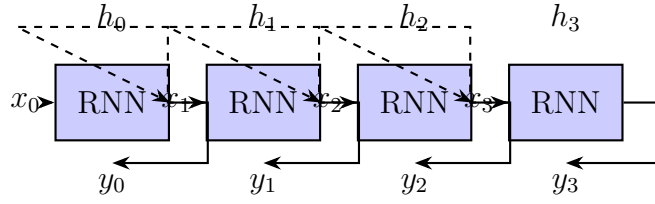
4 Recurrent Neural Networks (RNNs)

4.1 Introduction to Sequential Data

The Challenge: Traditional neural networks process fixed-size inputs independently. But many problems involve sequences:

- Natural language: Sentences are word sequences
- Time series: Stock prices over time
- Audio: Sound waves as time series

RNN Unrolled Through Time



RNN Equations: $h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$
 $y_t = W_{hy}h_t + b_y$ where h_t is hidden state at time t

Figure 12: RNNs process sequences one element at a time, maintaining hidden state

4.2 Vanilla RNN

Definition 4.1 (Vanilla RNN). For input sequence $\{x_1, x_2, \dots, x_T\}$, RNN computes:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

where h_t is hidden state at time t .

The Problem: Vanishing/exploding gradients when backpropagating through time.

4.3 Long Short-Term Memory (LSTM)

Solution: Gates control information flow to remember long-term dependencies.

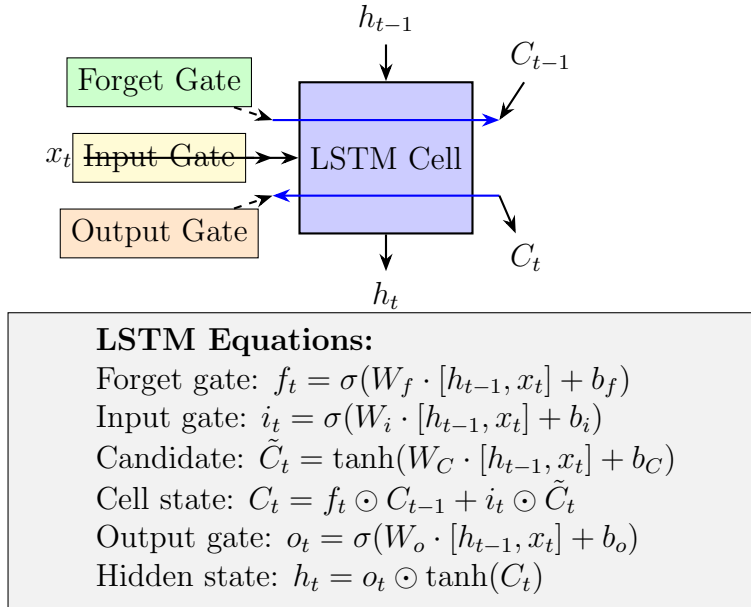


Figure 13: LSTM: Gates control what to forget, what new information to add, and what to output

4.4 Gated Recurrent Unit (GRU)

Simplified LSTM: Combines forget and input gates, merges cell and hidden states.

Definition 4.2 (GRU Equations).

$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) & (\text{Update gate}) \\
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) & (\text{Reset gate}) \\
 \tilde{h}_t &= \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b) & (\text{Candidate}) \\
 h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
 \end{aligned}$$

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 class VanillaRNN(nn.Module):
7     """Vanilla RNN implementation."""
8
9     def __init__(self, input_size, hidden_size, output_size):
10         super(VanillaRNN, self).__init__()
11
12         self.hidden_size = hidden_size
13
14         # Weights

```

```

15     self.W_xh = nn.Parameter(torch.randn(input_size, hidden_size
16                                     ) * 0.01)
17     self.W_hh = nn.Parameter(torch.randn(hidden_size,
18                                     hidden_size) * 0.01)
19     self.W_hy = nn.Parameter(torch.randn(hidden_size,
20                                     output_size) * 0.01)
21
22     # Biases
23     self.b_h = nn.Parameter(torch.zeros(hidden_size))
24     self.b_y = nn.Parameter(torch.zeros(output_size))
25
26     # Activation
27     self.tanh = nn.Tanh()
28
29 def forward(self, x, h_prev=None):
30     """
31     Forward pass through RNN.
32
33     Args:
34         x: Input sequence of shape (seq_len, batch_size,
35             input_size)
36         h_prev: Previous hidden state of shape (batch_size,
37             hidden_size)
38
39     Returns:
40         outputs: Sequence of outputs
41         h_last: Last hidden state
42     """
43     seq_len, batch_size, _ = x.shape
44
45     if h_prev is None:
46         h_prev = torch.zeros(batch_size, self.hidden_size,
47                             device=x.device)
48
49     outputs = []
50     h = h_prev
51
52     for t in range(seq_len):
53         # RNN update
54         h = self.tanh(
55             torch.matmul(x[t], self.W_xh) +
56             torch.matmul(h, self.W_hh) +
57             self.b_h
58         )
59
60         # Output
61         y = torch.matmul(h, self.W_hy) + self.b_y

```

```

56         outputs.append(y)
57
58         outputs = torch.stack(outputs, dim=0)
59         return outputs, h
60
61
62 class LSTMCell(nn.Module):
63     """Single LSTM cell."""
64
65     def __init__(self, input_size, hidden_size):
66         super(LSTMCell, self).__init__()
67
68         self.hidden_size = hidden_size
69
70         # Gates: input, forget, output, cell
71         self.W = nn.Linear(input_size + hidden_size, 4 * hidden_size
72                             )
73
74         # Initialize gates
75         self._init_weights()
76
77     def _init_weights(self):
78         """Initialize LSTM weights using Xavier initialization."""
79         for name, param in self.named_parameters():
80             if 'weight' in name:
81                 nn.init.xavier_uniform_(param)
82             elif 'bias' in name:
83                 # Set forget gate bias to 1 to help remembering
84                 nn.init.zeros_(param)
85                 param.data[self.hidden_size:2*self.hidden_size] =
86                     1.0
87
88     def forward(self, x, state):
89         """
90         Forward pass for single LSTM cell.
91
92         Args:
93             x: Input of shape (batch_size, input_size)
94             state: Tuple of (h_prev, C_prev)
95
96         Returns:
97             h: New hidden state
98             C: New cell state
99         """
100         h_prev, C_prev = state
101         batch_size = x.size(0)

```

```

101     # Concatenate input and previous hidden state
102     combined = torch.cat((x, h_prev), dim=1)
103
104     # Compute all gates
105     gates = self.W(combined)
106
107     # Split into input, forget, output, cell gates
108     i, f, o, g = torch.split(gates, self.hidden_size, dim=1)
109
110     # Apply activations
111     i = torch.sigmoid(i) # Input gate
112     f = torch.sigmoid(f) # Forget gate
113     o = torch.sigmoid(o) # Output gate
114     g = torch.tanh(g)    # Cell gate (candidate)
115
116     # Update cell state
117     C = f * C_prev + i * g
118
119     # Update hidden state
120     h = o * torch.tanh(C)
121
122     return h, C
123
124
125 class LSTM(nn.Module):
126     """Full LSTM network."""
127
128     def __init__(self, input_size, hidden_size, num_layers=1,
129                 dropout=0.0):
130         super(LSTM, self).__init__()
131
132         self.input_size = input_size
133         self.hidden_size = hidden_size
134         self.num_layers = num_layers
135         self.dropout = dropout
136
137         # Create LSTM layers
138         self.layers = nn.ModuleList()
139
140         for layer in range(num_layers):
141             input_dim = input_size if layer == 0 else hidden_size
142             self.layers.append(LSTMCell(input_dim, hidden_size))
143
144         if dropout > 0 and num_layers > 1:
145             self.dropout_layer = nn.Dropout(dropout)
146         else:
147             self.dropout_layer = None

```

```

147
148 def forward(self, x, state=None):
149     """
150     Forward pass through LSTM.
151
152     Args:
153         x: Input sequence of shape (seq_len, batch_size,
154             input_size)
155         state: Initial state tuple (h0, C0)
156
157     Returns:
158         outputs: Sequence of outputs
159         state: Final state tuple
160     """
161     seq_len, batch_size, _ = x.shape
162
163     # Initialize states if not provided
164     if state is None:
165         h = [torch.zeros(batch_size, self.hidden_size, device=x.
166             device)
167             for _ in range(self.num_layers)]
168         C = [torch.zeros(batch_size, self.hidden_size, device=x.
169             device)
170             for _ in range(self.num_layers)]
171     else:
172         h, C = state
173
174     outputs = []
175
176     for t in range(seq_len):
177         x_t = x[t]
178
179         new_h = []
180         new_C = []
181
182         for layer in range(self.num_layers):
183             # Get previous state for this layer
184             h_prev = h[layer]
185             C_prev = C[layer]
186
187             # LSTM cell forward
188             h_l, C_l = self.layers[layer](x_t, (h_prev, C_prev))
189
190             new_h.append(h_l)
191             new_C.append(C_l)
192
193             # Apply dropout to output (except last layer)

```

```

191         if self.dropout_layer is not None and layer < self.
192             num_layers - 1:
193             x_t = self.dropout_layer(h_l)
194         else:
195             x_t = h_l
196
197         h = new_h
198         C = new_C
199
200         outputs.append(h[-1]) # Output from last layer
201
202         outputs = torch.stack(outputs, dim=0)
203         state = (torch.stack(h, dim=0), torch.stack(C, dim=0))
204
205         return outputs, state
206
207 class GRUCell(nn.Module):
208     """Single GRU cell."""
209
210     def __init__(self, input_size, hidden_size):
211         super(GRUCell, self).__init__()
212
213         self.hidden_size = hidden_size
214
215         # Reset gate
216         self.W_xr = nn.Linear(input_size, hidden_size)
217         self.W_hr = nn.Linear(hidden_size, hidden_size, bias=False)
218
219         # Update gate
220         self.W_xz = nn.Linear(input_size, hidden_size)
221         self.W_hz = nn.Linear(hidden_size, hidden_size, bias=False)
222
223         # Candidate activation
224         self.W_xh = nn.Linear(input_size, hidden_size)
225         self.W_hh = nn.Linear(hidden_size, hidden_size, bias=False)
226
227     def forward(self, x, h_prev):
228         """
229         Forward pass for single GRU cell.
230
231         Args:
232             x: Input of shape (batch_size, input_size)
233             h_prev: Previous hidden state
234
235         Returns:
236             h: New hidden state

```

```

237     """
238     # Reset gate
239     r = torch.sigmoid(self.W_xr(x) + self.W_hr(h_prev))
240
241     # Update gate
242     z = torch.sigmoid(self.W_xz(x) + self.W_hz(h_prev))
243
244     # Candidate hidden state
245     h_tilde = torch.tanh(self.W_xh(x) + self.W_hh(r * h_prev))
246
247     # New hidden state
248     h = (1 - z) * h_prev + z * h_tilde
249
250     return h
251
252
253 class GRU(nn.Module):
254     """Full GRU network."""
255
256     def __init__(self, input_size, hidden_size, num_layers=1,
257                  dropout=0.0):
258         super(GRU, self).__init__()
259
260         self.input_size = input_size
261         self.hidden_size = hidden_size
262         self.num_layers = num_layers
263         self.dropout = dropout
264
265         # Create GRU layers
266         self.layers = nn.ModuleList()
267
268         for layer in range(num_layers):
269             input_dim = input_size if layer == 0 else hidden_size
270             self.layers.append(GRUCell(input_dim, hidden_size))
271
272         if dropout > 0 and num_layers > 1:
273             self.dropout_layer = nn.Dropout(dropout)
274         else:
275             self.dropout_layer = None
276
277     def forward(self, x, state=None):
278         """
279         Forward pass through GRU.
280
281         Args:
282             x: Input sequence of shape (seq_len, batch_size,
283                input_size)

```

```

282         state: Initial hidden state
283
284     Returns:
285         outputs: Sequence of outputs
286         h: Final hidden state
287     """
288     seq_len, batch_size, _ = x.shape
289
290     # Initialize hidden states if not provided
291     if state is None:
292         h = [torch.zeros(batch_size, self.hidden_size, device=x.
293                             device)
294               for _ in range(self.num_layers)]
295     else:
296         h = state
297
298     outputs = []
299
300     for t in range(seq_len):
301         x_t = x[t]
302
303         new_h = []
304
305         for layer in range(self.num_layers):
306             # Get previous hidden state for this layer
307             h_prev = h[layer]
308
309             # GRU cell forward
310             h_l = self.layers[layer](x_t, h_prev)
311
312             new_h.append(h_l)
313
314             # Apply dropout to output (except last layer)
315             if self.dropout_layer is not None and layer < self.
316                 num_layers - 1:
317                 x_t = self.dropout_layer(h_l)
318             else:
319                 x_t = h_l
320
321         h = new_h
322         outputs.append(h[-1]) # Output from last layer
323
324     outputs = torch.stack(outputs, dim=0)
325
326     return outputs, torch.stack(h, dim=0)

```

```

327 def demonstrate_rnn_capabilities():
328     """Demonstrate what different RNN architectures can learn."""
329
330     # Create synthetic sequence data
331     np.random.seed(42)
332     seq_len = 50
333     n_samples = 1000
334
335     # Task 1: Copy sequence
336     print("Task 1: Copy sequence (memorization test)")
337     X_copy = np.random.randint(0, 10, (n_samples, seq_len, 1))
338     y_copy = X_copy.copy() # Simply copy input
339
340     # Task 2: Reverse sequence
341     print("Task 2: Reverse sequence (order sensitivity test)")
342     X_reverse = np.random.randint(0, 10, (n_samples, seq_len, 1))
343     y_reverse = X_reverse[:, ::-1, :] # Reverse sequence
344
345     # Task 3: Temporal pattern (sum of last 3 elements)
346     print("Task 3: Sum of last 3 elements (temporal dependency test)")
347
348     X_sum = np.random.randn(n_samples, seq_len, 1)
349     y_sum = np.zeros((n_samples, seq_len, 1))
350     for i in range(seq_len):
351         y_sum[:, i, 0] = np.sum(X_sum[:, max(0, i-2):i+1, 0], axis
352                                 =1)
353
354     # Task 4: Parity (even/odd count)
355     print("Task 4: Parity of ones in binary sequence")
356     X_parity = np.random.randint(0, 2, (n_samples, seq_len, 1))
357     y_parity = np.zeros((n_samples, seq_len, 1))
358     for i in range(seq_len):
359         cumulative_sum = np.sum(X_parity[:, :i+1, 0], axis=1)
360         y_parity[:, i, 0] = cumulative_sum % 2
361
362     # Test models on these tasks
363     tasks = {
364         'Copy': (X_copy, y_copy),
365         'Reverse': (X_reverse, y_reverse),
366         'Sum3': (X_sum, y_sum),
367         'Parity': (X_parity, y_parity)
368     }
369
370     models = {
371         'RNN': VanillaRNN(input_size=1, hidden_size=32, output_size
372                             =1),
373         'LSTM': LSTM(input_size=1, hidden_size=32, num_layers=1),

```

```

371         'GRU': GRU(input_size=1, hidden_size=32, num_layers=1)
372     }
373
374     results = {}
375
376     for task_name, (X, y) in tasks.items():
377         print(f"\n{'='*60}")
378         print(f"Testing on task: {task_name}")
379         print(f'='*60)
380
381         task_results = {}
382
383         for model_name, model in models.items():
384             print(f"\n{model_name}:")
385
386             # Convert to PyTorch tensors
387             X_tensor = torch.FloatTensor(X[:100]) # Use subset for
388             # speed
389             y_tensor = torch.FloatTensor(y[:100])
390
391             # Forward pass
392             if model_name == 'RNN':
393                 outputs, _ = model(X_tensor.transpose(0, 1))
394                 outputs = outputs.transpose(0, 1)
395             else:
396                 outputs, _ = model(X_tensor.transpose(0, 1))
397                 outputs = outputs.transpose(0, 1)
398
399             # Calculate MSE
400             mse = torch.mean((outputs - y_tensor) ** 2).item()
401             print(f"    MSE: {mse:.6f}")
402
403             task_results[model_name] = mse
404
405         results[task_name] = task_results
406
407     # Plot results
408     fig, axes = plt.subplots(2, 2, figsize=(12, 10))
409     axes = axes.flatten()
410
411     for idx, (task_name, task_results) in enumerate(results.items()):
412         :
413         model_names = list(task_results.keys())
414         mse_values = list(task_results.values())
415
416         bars = axes[idx].bar(model_names, mse_values)
417         axes[idx].set_title(f'Task: {task_name}')

```

```

416     axes[idx].set_ylabel('MSE (lower is better)')
417     axes[idx].set_yscale('log')
418     axes[idx].grid(True, alpha=0.3, axis='y')
419
420     # Add value labels
421     for bar, value in zip(bars, mse_values):
422         axes[idx].text(bar.get_x() + bar.get_width()/2, bar.
423                        get_height(),
424                        f'{value:.2e}', ha='center', va='bottom')
425
426 plt.suptitle('RNN Architectures on Different Sequence Tasks',
427             fontsize=16)
428 plt.tight_layout()
429 plt.show()
430
431 # Visualize learning dynamics
432 print("\n" + "="*60)
433 print("Visualizing Gradient Flow in RNNs")
434 print("="*60)
435
436 def compute_gradient_norms(model, X, y, seq_lengths):
437     """Compute gradient norms for different sequence lengths."""
438     gradient_norms = []
439
440     for seq_len in seq_lengths:
441         # Truncate sequences
442         X_seq = X[:, :seq_len, :]
443         y_seq = y[:, :seq_len, :]
444
445         X_tensor = torch.FloatTensor(X_seq)
446         y_tensor = torch.FloatTensor(y_seq)
447
448         # Forward pass
449         if isinstance(model, VanillaRNN):
450             outputs, _ = model(X_tensor.transpose(0, 1))
451             outputs = outputs.transpose(0, 1)
452         else:
453             outputs, _ = model(X_tensor.transpose(0, 1))
454             outputs = outputs.transpose(0, 1)
455
456         # Compute loss
457         loss = torch.mean((outputs - y_tensor) ** 2)
458
459         # Backward pass
460         model.zero_grad()
461         loss.backward()

```

```

461         # Compute total gradient norm
462         total_norm = 0
463         for param in model.parameters():
464             if param.grad is not None:
465                 param_norm = param.grad.data.norm(2)
466                 total_norm += param_norm.item() ** 2
467         total_norm = total_norm ** 0.5
468
469         gradient_norms.append(total_norm)
470
471     return gradient_norms
472
473     # Test gradient norms for different sequence lengths
474     seq_lengths = [5, 10, 20, 30, 40, 50]
475
476     plt.figure(figsize=(10, 6))
477
478     for model_name, model in models.items():
479         gradient_norms = compute_gradient_norms(
480             model,
481             X_parity[:10], # Small batch
482             y_parity[:10],
483             seq_lengths
484         )
485
486         plt.plot(seq_lengths, gradient_norms, 'o-', label=model_name,
487                  linewidth=2)
488
489     plt.xlabel('Sequence Length')
490     plt.ylabel('Gradient Norm')
491     plt.title('Gradient Norms vs Sequence Length (Parity Task)')
492     plt.legend()
493     plt.grid(True, alpha=0.3)
494     plt.yscale('log')
495     plt.show()
496
497     return results
498
499     # Run demonstration
500     results = demonstrate_rnn_capabilities()

```

Listing 8: RNN, LSTM, and GRU Implementation

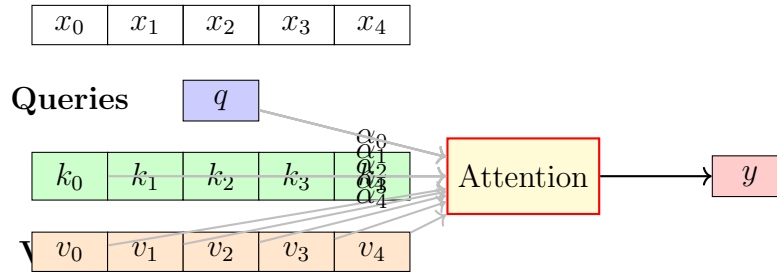
5 Attention Mechanisms and Transformers

5.1 The Attention Mechanism

The Problem with RNNs: Sequential processing prevents parallelization, and long-range dependencies are hard to capture.

Attention Solution: Allow the model to focus on relevant parts of the input when producing each output.

Input Sequence



$$\textbf{Attention: } \text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where $\alpha_i = \frac{\exp(q \cdot k_i / \sqrt{d})}{\sum_j \exp(q \cdot k_j / \sqrt{d})}$, $y = \sum_i \alpha_i v_i$

Figure 14: Attention mechanism: Compute weighted sum of values based on query-key similarity

5.2 Self-Attention

Key Insight: Instead of using different queries, keys, and values, use the same input sequence to generate all three.

Definition 5.1 (Self-Attention). For input $X \in \mathbb{R}^{n \times d}$:

$$Q = XW_Q \quad (\text{Queries})$$

$$K = XW_K \quad (\text{Keys})$$

$$V = XW_V \quad (\text{Values})$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

5.3 Multi-Head Attention

Idea: Run multiple attention mechanisms in parallel to capture different types of relationships.

Definition 5.2 (Multi-Head Attention).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

5.4 Transformer Architecture

Revolutionary Architecture: Completely attention-based, no recurrence.

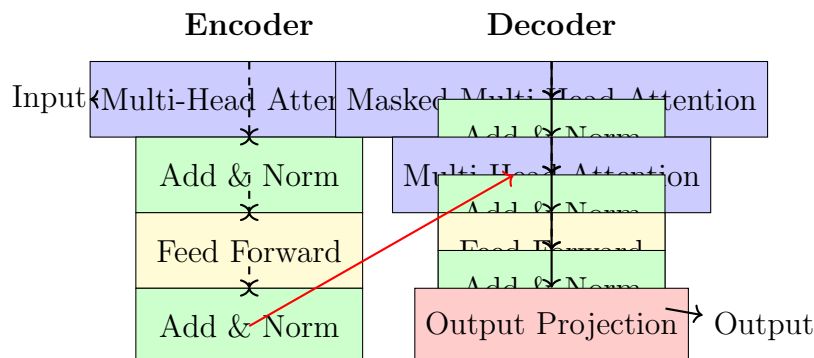


Figure 15: Transformer architecture: Encoder processes input, decoder generates output with attention to encoder

5.4.1 Positional Encoding

Problem: Attention is permutation-invariant. Need to encode position information.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5 import numpy as np
6
7 class PositionalEncoding(nn.Module):
8     """Positional encoding for Transformer."""
9
10     def __init__(self, d_model, max_len=5000):
11         super(PositionalEncoding, self).__init__()
12
13         # Create positional encoding matrix
14         pe = torch.zeros(max_len, d_model)
15         position = torch.arange(0, max_len, dtype=torch.float).
            unsqueeze(1)

```

```

16         div_term = torch.exp(torch.arange(0, d_model, 2).float() *
17                                (-math.log(10000.0) / d_model))
18
19         pe[:, 0::2] = torch.sin(position * div_term) # Even indices
20         pe[:, 1::2] = torch.cos(position * div_term) # Odd indices
21
22         pe = pe.unsqueeze(0) # Shape: (1, max_len, d_model)
23         self.register_buffer('pe', pe)
24
25     def forward(self, x):
26         """
27         Add positional encoding to input.
28
29         Args:
30             x: Tensor of shape (batch_size, seq_len, d_model)
31
32         Returns:
33             Tensor with positional encoding added
34         """
35         seq_len = x.size(1)
36         x = x + self.pe[:, :seq_len, :]
37         return x
38
39
40     class MultiHeadAttention(nn.Module):
41         """Multi-head attention mechanism."""
42
43         def __init__(self, d_model, num_heads, dropout=0.1):
44             super(MultiHeadAttention, self).__init__()
45
46             assert d_model % num_heads == 0, "d_model must be divisible
47                 by num_heads"
48
49             self.d_model = d_model
50             self.num_heads = num_heads
51             self.d_k = d_model // num_heads
52
53             # Linear projections for Q, K, V
54             self.W_q = nn.Linear(d_model, d_model)
55             self.W_k = nn.Linear(d_model, d_model)
56             self.W_v = nn.Linear(d_model, d_model)
57
58             # Output projection
59             self.W_o = nn.Linear(d_model, d_model)
60
61             # Dropout
62             self.dropout = nn.Dropout(dropout)

```

```

62
63     # Scale factor
64     self.scale = math.sqrt(self.d_k)
65
66     def forward(self, query, key, value, mask=None):
67         """
68         Forward pass for multi-head attention.
69
70         Args:
71             query: Query tensor of shape (batch_size, seq_len_q,
72                  d_model)
73             key: Key tensor of shape (batch_size, seq_len_k, d_model)
74             value: Value tensor of shape (batch_size, seq_len_v,
75                  d_model)
76             mask: Optional mask tensor
77
78         Returns:
79             Attention output and attention weights
80         """
81         batch_size = query.size(0)
82
83         # Linear projections and reshape for multi-head
84         Q = self.W_q(query).view(batch_size, -1, self.num_heads,
85                                   self.d_k).transpose(1, 2)
86         K = self.W_k(key).view(batch_size, -1, self.num_heads, self.
87                                 d_k).transpose(1, 2)
88         V = self.W_v(value).view(batch_size, -1, self.num_heads,
89                                   self.d_k).transpose(1, 2)
90
91         # Compute attention scores
92         scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale
93
94         # Apply mask if provided
95         if mask is not None:
96             scores = scores.masked_fill(mask == 0, -1e9)
97
98         # Apply softmax to get attention weights
99         attention_weights = F.softmax(scores, dim=-1)
100         attention_weights = self.dropout(attention_weights)
101
102         # Apply attention to values
103         context = torch.matmul(attention_weights, V)
104
105         # Reshape back to original shape
106         context = context.transpose(1, 2).contiguous().view(
107             batch_size, -1, self.d_model)

```

```

103         )
104
105         # Final linear projection
106         output = self.W_o(context)
107
108         return output, attention_weights
109
110
111 class PositionwiseFeedForward(nn.Module):
112     """Position-wise feed-forward network."""
113
114     def __init__(self, d_model, d_ff, dropout=0.1):
115         super(PositionwiseFeedForward, self).__init__()
116
117         self.linear1 = nn.Linear(d_model, d_ff)
118         self.linear2 = nn.Linear(d_ff, d_model)
119         self.dropout = nn.Dropout(dropout)
120         self.relu = nn.ReLU()
121
122     def forward(self, x):
123         """
124         Forward pass for feed-forward network.
125
126         Args:
127             x: Input tensor of shape (batch_size, seq_len, d_model)
128
129         Returns:
130             Output tensor
131         """
132         return self.linear2(self.dropout(self.relu(self.linear1(x))))
133
134
135 class TransformerEncoderLayer(nn.Module):
136     """Single transformer encoder layer."""
137
138     def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
139         super(TransformerEncoderLayer, self).__init__()
140
141         # Multi-head attention
142         self.self_attn = MultiHeadAttention(d_model, num_heads,
143                                             dropout)
144
145         # Feed-forward network
146         self.feed_forward = PositionwiseFeedForward(d_model, d_ff,
147                                                     dropout)

```

```

147     # Layer normalization
148     self.norm1 = nn.LayerNorm(d_model)
149     self.norm2 = nn.LayerNorm(d_model)
150
151     # Dropout
152     self.dropout1 = nn.Dropout(dropout)
153     self.dropout2 = nn.Dropout(dropout)
154
155     def forward(self, x, mask=None):
156         """
157         Forward pass for encoder layer.
158
159         Args:
160             x: Input tensor of shape (batch_size, seq_len, d_model)
161             mask: Optional mask tensor
162
163         Returns:
164             Output tensor
165         """
166         # Self-attention with residual connection
167         attn_output, _ = self.self_attn(x, x, x, mask)
168         x = x + self.dropout1(attn_output)
169         x = self.norm1(x)
170
171         # Feed-forward with residual connection
172         ff_output = self.feed_forward(x)
173         x = x + self.dropout2(ff_output)
174         x = self.norm2(x)
175
176         return x
177
178
179     class TransformerDecoderLayer(nn.Module):
180         """Single transformer decoder layer."""
181
182         def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
183             super(TransformerDecoderLayer, self).__init__()
184
185             # Masked multi-head attention
186             self.self_attn = MultiHeadAttention(d_model, num_heads,
187                                                  dropout)
188
189             # Encoder-decoder attention
190             self.cross_attn = MultiHeadAttention(d_model, num_heads,
191                                                  dropout)
192
193             # Feed-forward network

```

```

192         self.feed_forward = PositionwiseFeedForward(d_model, d_ff,
193                                                       dropout)
194
195         # Layer normalization
196         self.norm1 = nn.LayerNorm(d_model)
197         self.norm2 = nn.LayerNorm(d_model)
198         self.norm3 = nn.LayerNorm(d_model)
199
200         # Dropout
201         self.dropout1 = nn.Dropout(dropout)
202         self.dropout2 = nn.Dropout(dropout)
203         self.dropout3 = nn.Dropout(dropout)
204
205     def forward(self, x, encoder_output, src_mask=None, tgt_mask=
206                 None):
207         """
208         Forward pass for decoder layer.
209
210         Args:
211             x: Input tensor of shape (batch_size, seq_len, d_model)
212             encoder_output: Output from encoder
213             src_mask: Source mask for encoder-decoder attention
214             tgt_mask: Target mask for masked self-attention
215
216         Returns:
217             Output tensor
218         """
219         # Masked self-attention with residual connection
220         attn_output, _ = self.self_attn(x, x, x, tgt_mask)
221         x = x + self.dropout1(attn_output)
222         x = self.norm1(x)
223
224         # Encoder-decoder attention with residual connection
225         attn_output, _ = self.cross_attn(x, encoder_output,
226                                         encoder_output, src_mask)
227         x = x + self.dropout2(attn_output)
228         x = self.norm2(x)
229
230         # Feed-forward with residual connection
231         ff_output = self.feed_forward(x)
232         x = x + self.dropout3(ff_output)
233         x = self.norm3(x)
234
235         return x
236
237 class Transformer(nn.Module):

```

```

236     """Complete Transformer model."""
237
238     def __init__(self, src_vocab_size, tgt_vocab_size, d_model=512,
239                 num_heads=8,
240                 num_encoder_layers=6, num_decoder_layers=6, d_ff
241                     =2048,
242                 max_seq_length=100, dropout=0.1):
243         super(Transformer, self).__init__()
244
245         # Embeddings
246         self.src_embedding = nn.Embedding(src_vocab_size, d_model)
247         self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)
248
249         # Positional encoding
250         self.positional_encoding = PositionalEncoding(d_model,
251             max_seq_length)
252
253         # Encoder
254         self.encoder_layers = nn.ModuleList([
255             TransformerEncoderLayer(d_model, num_heads, d_ff,
256                 dropout)
257             for _ in range(num_encoder_layers)
258         ])
259
260         # Decoder
261         self.decoder_layers = nn.ModuleList([
262             TransformerDecoderLayer(d_model, num_heads, d_ff,
263                 dropout)
264             for _ in range(num_decoder_layers)
265         ])
266
267         # Final linear layer
268         self.output_projection = nn.Linear(d_model, tgt_vocab_size)
269
270         # Dropout
271         self.dropout = nn.Dropout(dropout)
272
273         # Initialize parameters
274         self._init_parameters()
275
276     def _init_parameters(self):
277         """Initialize transformer parameters."""
278         for p in self.parameters():
279             if p.dim() > 1:
280                 nn.init.xavier_uniform_(p)
281
282     def encode(self, src, src_mask):

```

```

278     """Encode source sequence."""
279     # Embedding + positional encoding
280     src_embedded = self.dropout(
281         self.positional_encoding(self.src_embedding(src))
282     )
283
284     # Pass through encoder layers
285     encoder_output = src_embedded
286     for layer in self.encoder_layers:
287         encoder_output = layer(encoder_output, src_mask)
288
289     return encoder_output
290
291 def decode(self, tgt, encoder_output, src_mask, tgt_mask):
292     """Decode target sequence."""
293     # Embedding + positional encoding
294     tgt_embedded = self.dropout(
295         self.positional_encoding(self.tgt_embedding(tgt))
296     )
297
298     # Pass through decoder layers
299     decoder_output = tgt_embedded
300     for layer in self.decoder_layers:
301         decoder_output = layer(decoder_output, encoder_output,
302                                src_mask, tgt_mask)
303
304     # Project to vocabulary size
305     output = self.output_projection(decoder_output)
306
307     return output
308
309 def forward(self, src, tgt, src_mask=None, tgt_mask=None):
310     """
311     Forward pass for transformer.
312
313     Args:
314         src: Source sequence of shape (batch_size, src_seq_len)
315         tgt: Target sequence of shape (batch_size, tgt_seq_len)
316         src_mask: Source mask
317         tgt_mask: Target mask
318
319     Returns:
320         Output logits
321     """
322     # Encode source
323     encoder_output = self.encode(src, src_mask)

```

```

324         # Decode target
325         output = self.decode(tgt, encoder_output, src_mask, tgt_mask
326                               )
327
328         return output
329
330     def generate_mask(self, src, tgt, pad_idx=0):
331         """Generate masks for source and target sequences."""
332         batch_size, src_len = src.size()
333         _, tgt_len = tgt.size()
334
335         # Source mask (padding mask)
336         src_mask = (src != pad_idx).unsqueeze(1).unsqueeze(2)
337
338         # Target mask (padding mask + look-ahead mask)
339         tgt_pad_mask = (tgt != pad_idx).unsqueeze(1).unsqueeze(2)
340         tgt_sub_mask = torch.tril(torch.ones(tgt_len, tgt_len)).
341             type_as(src).bool()
342         tgt_mask = tgt_pad_mask & tgt_sub_mask
343
344         return src_mask, tgt_mask
345
346 def demonstrate_transformer():
347     """Demonstrate transformer capabilities."""
348
349     # Create a simple translation-like task
350     np.random.seed(42)
351
352     # Vocabulary
353     vocab_size = 50
354     pad_idx = 0
355     sos_idx = 1 # Start of sequence
356     eos_idx = 2 # End of sequence
357
358     # Generate synthetic data
359     n_samples = 1000
360     max_seq_len = 20
361
362     def generate_sequence():
363         """Generate random sequence with start/end tokens."""
364         seq_len = np.random.randint(5, max_seq_len - 2)
365         sequence = np.random.randint(3, vocab_size, seq_len)
366         return [sos_idx] + sequence.tolist() + [eos_idx]
367
368     # Source sequences (input)
369     src_sequences = [generate_sequence() for _ in range(n_samples)]

```

```

369
370 # Target sequences (output) - reverse source as simple task
371 tgt_sequences = []
372 for src_seq in src_sequences:
373     # Remove SOS/EOS, reverse, add SOS/EOS
374     tgt_seq = src_seq[1:-1][::-1]
375     tgt_sequences.append([sos_idx] + tgt_seq + [eos_idx])
376
377 # Pad sequences
378 def pad_sequence(seq, max_len):
379     """Pad sequence to max length."""
380     return seq + [pad_idx] * (max_len - len(seq))
381
382 src_max_len = max(len(seq) for seq in src_sequences)
383 tgt_max_len = max(len(seq) for seq in tgt_sequences)
384
385 src_padded = [pad_sequence(seq, src_max_len) for seq in
386                src_sequences]
387 tgt_padded = [pad_sequence(seq, tgt_max_len) for seq in
388                tgt_sequences]
389
390 # Convert to tensors
391 src_tensor = torch.LongTensor(src_padded)
392 tgt_tensor = torch.LongTensor(tgt_padded)
393
394 print(f"Source sequences shape: {src_tensor.shape}")
395 print(f"Target sequences shape: {tgt_tensor.shape}")
396 print(f"Vocabulary size: {vocab_size}")
397
398 # Create transformer model
399 transformer = Transformer(
400     src_vocab_size=vocab_size,
401     tgt_vocab_size=vocab_size,
402     d_model=128, # Smaller for demonstration
403     num_heads=8,
404     num_encoder_layers=3,
405     num_decoder_layers=3,
406     d_ff=512,
407     max_seq_length=max(src_max_len, tgt_max_len),
408     dropout=0.1
409 )
410
411 print(f"\nTransformer model created:")
412 print(f"    d_model: {128}")
413 print(f"    num_heads: {8}")
414 print(f"    encoder layers: {3}")
415 print(f"    decoder layers: {3}")

```

```

414
415 # Generate masks
416 src_mask, tgt_mask = transformer.generate_mask(
417     src_tensor[:1], # First batch only
418     tgt_tensor[:1],
419     pad_idx=pad_idx
420 )
421
422 print(f"\nMask shapes:")
423 print(f"    Source mask: {src_mask.shape}")
424 print(f"    Target mask: {tgt_mask.shape}")
425
426 # Forward pass
427 with torch.no_grad():
428     output = transformer(
429         src_tensor[:1], # First batch
430         tgt_tensor[:1, :-1], # Teacher forcing
431         src_mask,
432         tgt_mask[:, :, :-1, :-1] # Adjust mask for shifted
433             target
434     )
435
436 print(f"\nOutput shape: {output.shape}")
437 print(f"Expected: (batch_size, tgt_seq_len-1, vocab_size)")
438
439 # Visualize attention weights
440 print("\n" + "="*60)
441 print("Visualizing Attention Weights")
442 print("="*60)
443
444 # Get attention weights from first encoder layer
445 encoder_layer = transformer.encoder_layers[0]
446
447 # Create a dummy forward pass to get attention weights
448 with torch.no_grad():
449     # Embedding + positional encoding
450     src_embedded = transformer.dropout(
451         transformer.positional_encoding(transformer.
452             src_embedding(src_tensor[:1]))
453     )
454
455     # Forward through first encoder layer
456     # We need to modify the forward method to return attention
457     # weights
458     # For simplicity, let's create a custom forward
459
460     # Create a sample sequence

```

```

458     sample_src = torch.LongTensor([[1, 3, 5, 7, 9, 2]]) # SOS,
         tokens, EOS
459     sample_src_mask, _ = transformer.generate_mask(sample_src,
         sample_src)
460
461     # Manual attention computation for visualization
462     batch_size, seq_len = sample_src.shape
463     d_model = transformer.src_embedding.embedding_dim
464
465     # Embed
466     src_emb = transformer.src_embedding(sample_src)
467     src_emb = transformer.positional_encoding(src_emb)
468
469     # Create multi-head attention
470     mha = MultiHeadAttention(d_model=d_model, num_heads=2)
471
472     # Compute attention
473     attn_output, attn_weights = mha(src_emb, src_emb, src_emb,
         sample_src_mask)
474
475     print(f"Attention weights shape: {attn_weights.shape}")
476     print(f"Expected: (batch_size, num_heads, seq_len, seq_len)"
         )
477
478     # Visualize attention matrix
479     plt.figure(figsize=(10, 8))
480
481     # Plot attention weights for first head
482     attn_matrix = attn_weights[0, 0].cpu().numpy()
483
484     plt.imshow(attn_matrix, cmap='viridis', interpolation='nearest')
485     plt.colorbar(label='Attention Weight')
486     plt.xlabel('Key Position')
487     plt.ylabel('Query Position')
488     plt.title('Attention Weights (First Head)')
489
490     # Add text annotations
491     for i in range(attn_matrix.shape[0]):
492         for j in range(attn_matrix.shape[1]):
493             plt.text(j, i, f'{attn_matrix[i, j]:.2f}',
494                     ha='center', va='center', color='white' if
495                         attn_matrix[i, j] > 0.5 else 'black')
496
497     plt.tight_layout()
498     plt.show()
499
500     # Demonstrate different attention patterns

```

```

500 print("\n" + "="*60)
501 print("Different Attention Patterns")
502 print("="*60)
503
504 patterns = {
505     'Causal (Decoder)': np.tril(np.ones((6, 6))),
506     'Full (Encoder)': np.ones((6, 6)),
507     'Local (Window)': np.triu(np.tril(np.ones((6, 6)), k=2), k
508         =-2),
509     'Dilated': np.eye(6) + np.eye(6, k=2) + np.eye(6, k=-2),
510     'Random': np.random.rand(6, 6),
511     'Diagonal': np.eye(6)
512 }
513
514 fig, axes = plt.subplots(2, 3, figsize=(12, 8))
515 axes = axes.flatten()
516
517 for idx, (name, pattern) in enumerate(patterns.items()):
518     im = axes[idx].imshow(pattern, cmap='viridis')
519     axes[idx].set_title(name)
520     axes[idx].set_xlabel('Key Position')
521     axes[idx].set_ylabel('Query Position')
522     plt.colorbar(im, ax=axes[idx])
523
524 plt.suptitle('Different Attention Patterns', fontsize=16)
525 plt.tight_layout()
526 plt.show()
527
528 return transformer, src_tensor, tgt_tensor
529
530 # Run transformer demonstration
531 transformer, src_data, tgt_data = demonstrate_transformer()

```

Listing 9: Transformer Implementation

5.5 BERT and GPT Architectures

5.5.1 BERT (Bidirectional Encoder Representations from Transformers)

Key Innovations:

- Bidirectional training (masked language modeling)
- Next sentence prediction task
- Pre-training on large corpus, fine-tuning for specific tasks

5.5.2 GPT (Generative Pre-trained Transformer)

Key Innovations:

- Unidirectional (causal) attention
- Generative pre-training
- Autoregressive language modeling

```
1 import torch
2 import torch.nn as nn
3 import math
4
5 class BERTEmbedding(nn.Module):
6     """BERT embedding layer (token + position + segment)."""
7
8     def __init__(self, vocab_size, d_model, max_len=512, n_segments
9         =2):
10
11         super(BERTEmbedding, self).__init__()
12
13         self.token_embedding = nn.Embedding(vocab_size, d_model)
14         self.position_embedding = nn.Embedding(max_len, d_model)
15         self.segment_embedding = nn.Embedding(n_segments, d_model)
16
17         self.layer_norm = nn.LayerNorm(d_model)
18         self.dropout = nn.Dropout(0.1)
19
20     def forward(self, x, segment_labels):
21         """
22         Forward pass for BERT embedding.
23
24         Args:
25             x: Input tokens of shape (batch_size, seq_len)
26             segment_labels: Segment labels of shape (batch_size,
27                 seq_len)
28
29         Returns:
30             Combined embeddings
31         """
32         seq_len = x.size(1)
33         positions = torch.arange(seq_len, dtype=torch.long, device=x
34             .device)
35         positions = positions.unsqueeze(0).expand_as(x)
36
37         token_emb = self.token_embedding(x)
38         position_emb = self.position_embedding(positions)
39         segment_emb = self.segment_embedding(segment_labels)
```

```

37     embeddings = token_emb + position_emb + segment_emb
38     embeddings = self.layer_norm(embeddings)
39     embeddings = self.dropout(embeddings)
40
41     return embeddings
42
43
44 class BERT(nn.Module):
45     """Simplified BERT model."""
46
47     def __init__(self, vocab_size, d_model=768, num_heads=12,
48                 num_layers=12, d_ff=3072, max_len=512):
49         super(BERT, self).__init__()
50
51         # Embedding
52         self.embedding = BERTEmbedding(vocab_size, d_model, max_len)
53
54         # Transformer encoder layers
55         self.encoder_layers = nn.ModuleList([
56             TransformerEncoderLayer(d_model, num_heads, d_ff)
57             for _ in range(num_layers)
58         ])
59
60         # Pooler (for classification)
61         self.pooler = nn.Sequential(
62             nn.Linear(d_model, d_model),
63             nn.Tanh()
64         )
65
66     def forward(self, x, segment_labels, attention_mask=None):
67         """
68         Forward pass for BERT.
69
70         Args:
71             x: Input tokens
72             segment_labels: Segment labels
73             attention_mask: Attention mask
74
75         Returns:
76             Sequence output and pooled output
77         """
78         # Get embeddings
79         embeddings = self.embedding(x, segment_labels)
80
81         # Apply transformer encoder layers
82         hidden_states = embeddings
83         for layer in self.encoder_layers:

```

```

84         hidden_states = layer(hidden_states, attention_mask)
85
86     # Pooled output (CLS token)
87     pooled_output = self.pooler(hidden_states[:, 0])
88
89     return hidden_states, pooled_output
90
91
92 class GPT(nn.Module):
93     """Simplified GPT model."""
94
95     def __init__(self, vocab_size, d_model=768, num_heads=12,
96                 num_layers=12, d_ff=3072, max_len=512):
97         super(GPT, self).__init__()
98
99         # Token and position embeddings
100         self.token_embedding = nn.Embedding(vocab_size, d_model)
101         self.position_embedding = nn.Embedding(max_len, d_model)
102         self.dropout = nn.Dropout(0.1)
103
104         # Transformer decoder layers (causal attention)
105         self.decoder_layers = nn.ModuleList([
106             TransformerDecoderLayer(d_model, num_heads, d_ff)
107             for _ in range(num_layers)
108         ])
109
110         # Final layer norm and output projection
111         self.layer_norm = nn.LayerNorm(d_model)
112         self.output_projection = nn.Linear(d_model, vocab_size, bias
113                                             =False)
114
115         # Tie weights (embedding and output projection)
116         self.output_projection.weight = self.token_embedding.weight
117
118         self.max_len = max_len
119
120     def forward(self, x, past_key_values=None):
121         """
122         Forward pass for GPT.
123
124         Args:
125             x: Input tokens of shape (batch_size, seq_len)
126             past_key_values: Past key-values for generation
127
128         Returns:
129             Logits and updated past key-values
130         """

```

```

130     batch_size, seq_len = x.size()
131
132     # Create position indices
133     if past_key_values is None:
134         past_length = 0
135     else:
136         past_length = past_key_values[0][0].size(-2)
137
138     position_ids = torch.arange(past_length, past_length +
139                                seq_len,
140                                dtype=torch.long, device=x.device
141                                )
142     position_ids = position_ids.unsqueeze(0).expand(batch_size,
143                                                    -1)
144
145     # Get embeddings
146     token_emb = self.token_embedding(x)
147     position_emb = self.position_embedding(position_ids)
148     hidden_states = self.dropout(token_emb + position_emb)
149
150     # Create causal mask
151     causal_mask = torch.tril(torch.ones(seq_len, seq_len)).to(x.
152                                                                device)
153     causal_mask = causal_mask.unsqueeze(0).unsqueeze(0)
154
155     # Apply decoder layers
156     present_key_values = []
157     for i, layer in enumerate(self.decoder_layers):
158         if past_key_values is not None:
159             past_layer_key_values = past_key_values[i]
160         else:
161             past_layer_key_values = None
162
163         # For simplicity, we'll use a modified forward
164         # In practice, GPT uses decoder-only architecture
165
166         # Self-attention
167         attn_output, attn_weights, present = self._attention(
168             hidden_states, causal_mask, past_layer_key_values
169         )
170         hidden_states = hidden_states + attn_output
171         hidden_states = self.layer_norm(hidden_states)
172
173         # Feed-forward
174         ff_output = self._feed_forward(hidden_states)
175         hidden_states = hidden_states + ff_output
176         hidden_states = self.layer_norm(hidden_states)

```

```

173         present_key_values.append(present)
174
175     # Project to vocabulary
176     logits = self.output_projection(hidden_states)
177
178     return logits, present_key_values
179
180
181 def _attention(self, x, mask, past_key_values):
182     """Simplified attention for GPT."""
183     # This is a simplified version
184     # In practice, use MultiHeadAttention with caching
185     return x, None, None
186
187 def _feed_forward(self, x):
188     """Simplified feed-forward for GPT."""
189     # This is a simplified version
190     return x
191
192
193 def compare_bert_gpt():
194     """Compare BERT and GPT architectures."""
195
196     print("="*60)
197     print("BERT vs GPT Architecture Comparison")
198     print("="*60)
199
200     vocab_size = 30000
201     seq_len = 128
202     batch_size = 2
203
204     # Create sample inputs
205     tokens = torch.randint(0, vocab_size, (batch_size, seq_len))
206     segment_labels = torch.randint(0, 2, (batch_size, seq_len))
207
208     # Create models
209     bert = BERT(vocab_size, d_model=768, num_layers=12, num_heads
210                =12)
211     gpt = GPT(vocab_size, d_model=768, num_layers=12, num_heads=12)
212
213     print(f"\nInput shape: {tokens.shape}")
214
215     # BERT forward
216     print("\nBERT:")
217     print("-" * 30)
218     with torch.no_grad():
219         hidden_states, pooled_output = bert(tokens, segment_labels)

```

```

219     print(f"Hidden states shape: {hidden_states.shape}")
220     print(f"Pooled output shape: {pooled_output.shape}")
221     print(f"Total parameters: {sum(p.numel() for p in bert.
        parameters()):,}")
222
223     # GPT forward
224     print("\nGPT:")
225     print("-" * 30)
226     with torch.no_grad():
227         logits, _ = gpt(tokens)
228         print(f"Logits shape: {logits.shape}")
229         print(f"Total parameters: {sum(p.numel() for p in gpt.
        parameters()):,}")
230
231     # Comparison table
232     print("\n" + "="*60)
233     print("Architecture Differences:")
234     print("="*60)
235
236     differences = [
237         ("Training Objective", "Masked LM + NSP", "Causal LM"),
238         ("Attention", "Bidirectional", "Unidirectional (Causal)"),
239         ("Position Encoding", "Absolute", "Absolute"),
240         ("Typical Use", "Classification, QA", "Generation"),
241         ("Context", "Full sequence", "Left context only"),
242         ("Fine-tuning", "Task-specific heads", "Prompt-based or fine
        -tuning")
243     ]
244
245     print("\n{:<25} {:<20} {:<20}".format(
246         "Aspect", "BERT", "GPT"))
247     print("-" * 65)
248
249     for aspect, bert_desc, gpt_desc in differences:
250         print("{:<25} {:<20} {:<20}".format(
251             aspect, bert_desc, gpt_desc))
252
253     # Visualization of attention patterns
254     print("\n" + "="*60)
255     print("Attention Patterns Visualization")
256     print("="*60)
257
258     # Create attention pattern diagrams
259     seq_length = 8
260
261     # BERT pattern (bidirectional)
262     bert_pattern = torch.ones(seq_length, seq_length)

```

```

263 # GPT pattern (causal)
264 gpt_pattern = torch.tril(torch.ones(seq_length, seq_length))
265
266
267 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
268
269 # BERT
270 im1 = axes[0].imshow(bert_pattern, cmap='viridis')
271 axes[0].set_title('BERT: Bidirectional Attention')
272 axes[0].set_xlabel('Key Position')
273 axes[0].set_ylabel('Query Position')
274 plt.colorbar(im1, ax=axes[0])
275
276 # Add text
277 for i in range(seq_length):
278     for j in range(seq_length):
279         axes[0].text(j, i, '1', ha='center', va='center', color=
            'white')
280
281 # GPT
282 im2 = axes[1].imshow(gpt_pattern, cmap='viridis')
283 axes[1].set_title('GPT: Causal Attention')
284 axes[1].set_xlabel('Key Position')
285 axes[1].set_ylabel('Query Position')
286 plt.colorbar(im2, ax=axes[1])
287
288 # Add text
289 for i in range(seq_length):
290     for j in range(seq_length):
291         if j <= i:
292             axes[1].text(j, i, '1', ha='center', va='center',
                color='white')
293         else:
294             axes[1].text(j, i, '0', ha='center', va='center',
                color='black')
295
296 plt.tight_layout()
297 plt.show()
298
299 return bert, gpt
300
301 # Compare BERT and GPT
302 bert_model, gpt_model = compare_bert_gpt()

```

Listing 10: BERT-style Implementation

6 Advanced Topics in Deep Learning

6.1 Generative Models

6.1.1 Variational Autoencoders (VAEs)

Idea: Learn latent representation and generate new samples.

Definition 6.1 (VAE Objective).

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$$

where:

- *First term: Reconstruction loss*
- *Second term: KL divergence (regularization)*

6.1.2 Generative Adversarial Networks (GANs)

Idea: Two networks compete - generator creates fake samples, discriminator tries to detect fakes.

Definition 6.2 (GAN Objective).

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

6.2 Self-Supervised Learning

The Future: Learn from unlabeled data by creating pretext tasks.

- **Contrastive Learning:** SimCLR, MoCo
- **Masked Modeling:** BERT, MAE
- **Next Token Prediction:** GPT

6.3 Model Optimization and Deployment

6.3.1 Quantization

Reduce precision of weights/activations to speed up inference.

- FP32 $\rightarrow$ FP16 (half precision)
- FP32 $\rightarrow$ INT8 (8-bit integers)
- Mixed precision training

6.3.2 Pruning

Remove unimportant weights to reduce model size.

- Magnitude-based pruning
- Structured pruning
- Lottery ticket hypothesis

6.3.3 Knowledge Distillation

Train small student model to mimic large teacher model.

6.4 Ethical Considerations

Responsible AI Practices

1. **Bias and Fairness:** Ensure models don't discriminate
2. **Transparency:** Make model decisions explainable
3. **Privacy:** Protect sensitive data with techniques like differential privacy
4. **Security:** Guard against adversarial attacks
5. **Environmental Impact:** Consider computational costs and carbon footprint

7 Conclusion

7.1 Key Takeaways

1. **Start Simple:** Begin with MLPs before moving to CNNs/RNNs/Transformers
2. **Understand Fundamentals:** Backpropagation, optimization, regularization
3. **Choose Right Architecture:**
 - Images → CNNs
 - Sequences → RNNs/Transformers
 - Generation → VAEs/GANs
4. **Regularize Properly:** Dropout, batch norm, early stopping
5. **Monitor Training:** Track loss curves, check for overfitting

7.2 Current Trends

Trend	Description	Key Models
Large Language Models	Scale transformers to billions of parameters	GPT-4, PaLM, LLaMA
Multimodal AI	Combine text, image, audio, video	CLIP, DALL-E, Flamingo
Efficient Models	Reduce computation while maintaining performance	EfficientNet, MobileNet, DistilBERT
Self-Supervised Learning	Learn from unlabeled data	BERT, MAE, SimCLR
Neural Architecture Search	Automatically discover optimal architectures	NASNet, EfficientNet

Table 3: Current trends in deep learning

7.3 Learning Resources

- **Courses:**

- Stanford CS231n (Computer Vision)
- Stanford CS224n (NLP)
- Fast.ai (Practical Deep Learning)

- **Books:**

- "Deep Learning" by Goodfellow, Bengio, Courville
- "Pattern Recognition and Machine Learning" by Bishop

- **Papers:**

- arXiv.org
- Papers with Code

- **Frameworks:**

- PyTorch (research, flexibility)
- TensorFlow (production, deployment)
- JAX (high-performance, functional)

7.4 Final Advice

Deep Learning Best Practices

1. **Reproducibility:** Set random seeds, document hyperparameters
2. **Version Control:** Track code, data, and model versions
3. **Experimentation:** Use tools like Weights & Biases, MLflow
4. **Validation:** Always use hold-out validation set
5. **Interpretability:** Understand why models make predictions
6. **Continual Learning:** Stay updated with latest research
7. **Practical Focus:** Solve real problems, not just optimize metrics

Remember: Deep learning is a rapidly evolving field. The fundamentals remain important, but be prepared to learn new architectures and techniques. Focus on developing intuition, not just memorizing equations. Build projects, read papers, and collaborate with others.

The Journey Ahead: From here, you can specialize in:

- Computer Vision: Object detection, segmentation, video analysis
- Natural Language Processing: Translation, summarization, dialogue systems
- Reinforcement Learning: Game playing, robotics, optimization
- Generative AI: Text-to-image, music generation, creative applications
- Multimodal AI: Combining different types of data

Keep learning, keep building, and most importantly - have fun with deep learning!